

A Demonstration of Quantum Circuit Implementation for Obstacle Flow Using Carleman-Linearized Lattice Boltzmann Method

Kazumasa Ueno,^{1,2*} Keita Kanno,¹ and Yasunori Lee¹

¹ *QunaSys, Bunkyo, Tokyo 113-0001, Japan*

² *Department of Earth and Planetary Science,
The University of Tokyo, Bunkyo, Tokyo 113-0033, Japan*

Fluid simulations, especially at high Reynolds numbers, are computationally expensive on classical computers, making them promising application targets for quantum computing. Recent studies have combined the lattice Boltzmann method (LBM) with Carleman linearization to design quantum algorithms for computational fluid dynamics (CFD). However, practical quantum-circuit implementations of these algorithms that incorporate *non-periodic* boundary conditions have not been fully explored. In this work, we implement a quantum algorithm for two-dimensional linearized fluid flow around an obstacle, using block-encoding of the linear-system matrix and quantum singular value transformation (QSVT) to solve it. Inflow, outflow, and no-slip boundary conditions are formulated as sparse matrix operations and efficiently embedded into quantum circuits using index-value encoding. We demonstrate logarithmic scaling of the required numbers of qubits and gates with respect to the number of lattice points, suggesting the potential feasibility of quantum-computational fluid dynamics simulations.

*kazumasa-e67@eps.s.u-tokyo.ac.jp

Contents

1	Introduction	1
2	Methods	4
2.1	Lattice Boltzmann method (LBM)	4
2.1.1	Preliminaries	4
2.1.2	Description	4
2.1.3	Implementation of boundary conditions	6
2.2	Quantum lattice Boltzmann method (QLBM)	8
2.2.1	LBE in vector matrix form	8
2.2.2	Carleman linearization	11
2.2.3	Time evolution and final state idling	13
2.3	Quantum Circuit	14
2.3.1	State vector and velocity encoding	14
2.3.2	Block-encoding of matrix $A^{(1)}$	14
2.3.3	Block-encoding of matrix L	19
2.3.4	QLSA by QSVT	21
3	Results	23
3.1	Effective condition number of the matrix L	23
3.2	Simulation results of flow around an obstacle	24
3.2.1	Quantum circuit simulation	25
3.2.2	Classical emulation via the Clenshaw algorithm	26
3.3	Quantum computational costs of U_L	29
3.4	Fault-tolerant T-gate cost estimation	30
4	Discussion	31
5	Conclusion	33
A	Notation	35

1 Introduction

Research on the application of quantum computing to classical problems has become increasingly active in recent years. A prominent target is computational fluid dynamics (CFD), which is indispensable in many fields, ranging from automotive engineering to weather forecasting. Since high-fidelity numerical simulations of fluid flows on classical computers, especially at high Reynolds numbers, are prohibitively costly, quantum computing holds promise for breakthroughs in this area.

Fluid flows are typically governed by nonlinear partial differential equations (PDEs), such as the Navier–Stokes equations. However, directly solving these equations is not the only way to simulate them. One alternative way is to use the *lattice Boltzmann method* (LBM), which is a mesoscopic method that represents fluid states by a distribution function $f_q(\mathbf{x}, t)$ of fictitious particles moving on a discrete grid with discrete velocities, and models the dynamics by solving the Boltzmann kinetic equation in $f_q(\mathbf{x}, t)$. Although the Boltzmann equation is still nonlinear, the LBM is notable for its structural feature that is advantageous for quantum implementations: it separates nonlinearity and nonlocality. Specifically, the collision step (relaxation toward equilibrium) is nonlinear but local to each lattice site, whereas the streaming step (propagation) is linear and nonlocal. This makes the nonlinearity in the LBM formulation more amenable to *Carleman linearization* than that in the original Navier-Stokes formulation.

For quantum algorithms for solving time evolution equations, there are two main approaches. The first approach is to implement the quantum circuit corresponding to a single-time-step evolution and apply it repeatedly to the quantum state. For the LBM case, the collision and streaming operators can be *block-encoded* into quantum circuits based on their representation as linear combinations of unitaries (LCU) [MSL⁺15, Bud21, SS23] (while costly) or their structure [SSR⁺25] (with the help of linearization), for instance. However, this approach in general suffers from exponential decay in the amplitude of the target quantum state with respect to the number of time steps, which poses a significant challenge.

This partly motivates the second approach, which embeds the full time evolution into a single linear system and solves it all at once using a quantum linear system algorithm (QLSA). For the LBM case, the discrete-time lattice Boltzmann equation (LBE) can be embedded into a linear system by Carleman linearization, and the matrix to be inverted can be block-encoded into quantum circuits based on its structure [LYW⁺23, PKJ⁺24].

A notable recent contribution in this line is the work of [JKL⁺25a, JKL⁺25b], which developed an end-to-end quantum algorithm for nonlinear fluid dynamics and achieved bounded quantum advantage under specific conditions, validated on nontrivial incompressible flow benchmarks.

Despite this progress, there are two key aspects which remain to be addressed in more depth. First, the dependence of the condition number of the linear system on the problem parameters has not been fully characterized. In the analysis of [JKL⁺25a], all the simulation parameters (i.e., the number of lattice points N , the number of time steps N_t , and the relaxation time τ) are simultaneously fixed as functions of Reynolds number through a specific parameter selection, and the scaling of the condition number is analyzed primarily with respect to Reynolds number. This somewhat obscures the independent contributions of N_t and N to the condition number, which is important for identifying the regime in which the quantum algorithm may be advantageous. Also, while [JKL⁺25a] constructed explicit quantum circuits for the collision and streaming operators in the periodic setting and [JKL⁺25b] extended the algorithm to incorporate nontrivial boundary conditions at the block-encoding level, a complete gate-level implementation of the block-encoding of system matrices that include realistic boundary conditions (e.g., inflow, outflow, and no-slip walls) and obstacle geometries is still lacking.

In this work, we take a step toward filling these gaps by developing a quantum algorithm for simulating flows around an obstacle using first-order Carleman-linearized LBM and a QLSA based on quantum singular value transformation (QSVT). Following the discrete-time formulation of the LBE, we express the full time-discretized update (collision and streaming together with boundary conditions) as a sparse matrix acting on the vector of distribution functions. The resulting linear system is solved via matrix inversion based on QSVT. We analyze the condition number of the linear system and identify that it is primarily governed by the number of time steps N_t , with weak dependence on the spatial resolution. This finding suggests that the quantum algorithm may be most advantageous for problems requiring high spatial resolution but only moderate temporal extent. Furthermore, we decompose the block-encoding of this matrix into four oracle quantum circuits (O_{setBC} , $O_{\text{collision}}$, $O_{\text{streaming}}$, O_{unsetBC}) that mirror the collision-streaming factorization of the evolution matrix, and describe each oracle’s construction in detail. We provide explicit gate-level resource estimates (qubit counts and gate counts) for representative obstacle-flow configurations.

The rest of the paper is organized as follows:

- In Section 2, we describe the methods used in this study, including the formulation of boundary conditions and the implementation of the quantum circuits.
- In Section 3, we present the results of our simulations, including the condition number analysis and the required numbers of qubits and gates.
- In Section 4, we discuss the implications of our findings and possible future directions.
- In Section 5, we summarize our conclusions.

The notation used in this paper is summarized in Appendix A.

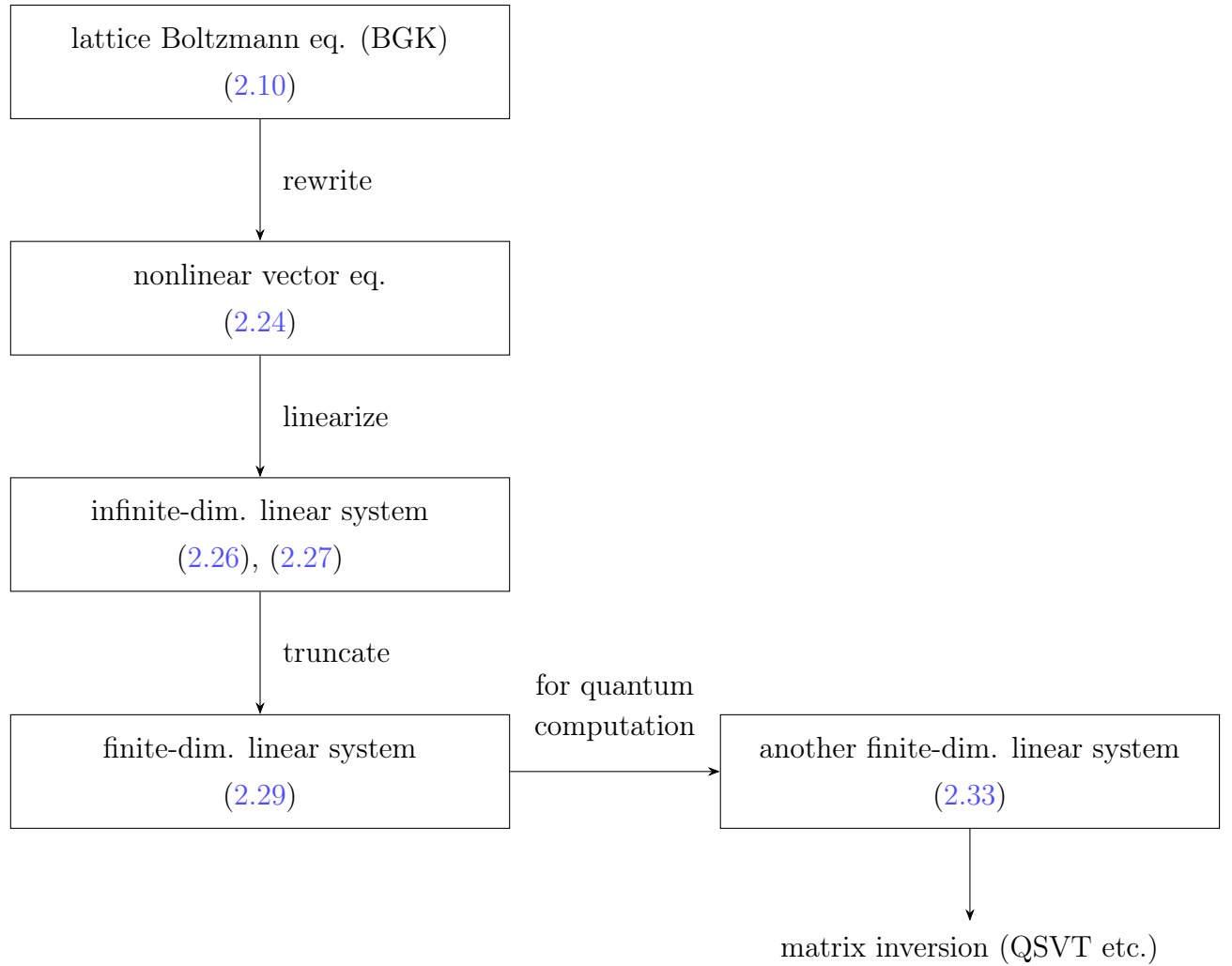


Figure 1: High-level overview of the quantum lattice Boltzmann method.

2 Methods

2.1 Lattice Boltzmann method (LBM)

Here we briefly summarize the lattice Boltzmann method. For more details, the reader is referred to standard textbooks, e.g., [KKK⁺16, Suc18].

2.1.1 Preliminaries

The two key dimensionless parameters governing a fluid flow are the Reynolds number

$$\text{Re} = \frac{Ul}{\nu} \quad (2.1)$$

and the Mach number

$$\text{Ma} = \frac{U}{c_s} \quad (2.2)$$

where U is the characteristic velocity, l is the characteristic length, ν is the kinematic viscosity, and c_s is the speed of sound. In the present study, we take U to be the inflow speed and l to be the channel width.

2.1.2 Description

In the LBM, fluid states are represented by distribution functions $f_q(\mathbf{x}_n, t)$ at lattice node $\mathbf{x}_n$ and time t with discrete velocity $\mathbf{c}_q$, from which the macroscopic density ρ and the macroscopic velocity $\mathbf{u}$ can be obtained as

$$\rho(\mathbf{x}_n, t) = \sum_q f_q(\mathbf{x}_n, t), \quad (2.3)$$

$$\rho \mathbf{u}(\mathbf{x}_n, t) = \sum_q f_q(\mathbf{x}_n, t) \mathbf{c}_q. \quad (2.4)$$

The discrete velocity set $\{\mathbf{c}_q\}$ is chosen so that it satisfies isotropy and moment-matching conditions required to recover the original Navier–Stokes equations. Such velocity sets in D spatial dimensions with Q discrete velocities are called DDQQ models. In this study, we consider an $N_x \times N_y$ square lattice and use the D2Q9 model whose discrete velocity vectors and associated weights are shown in Fig. 2 and Table 1, and the speed of sound is given by

$$c_s = \frac{1}{\sqrt{3}} \frac{\Delta x}{\Delta t} \quad (2.5)$$

for the lattice spacing Δx and the time step Δt .

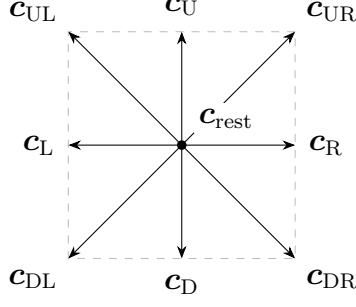


Figure 2: Unit discrete velocity vectors of the D2Q9 lattice. The vectors $\mathbf{c}_R$, $\mathbf{c}_U$, $\dots$, $\mathbf{c}_{DR}$ correspond to the eight directions connecting a node to its nearest neighbors, and $\mathbf{c}_{\text{rest}}$ denotes the rest velocity ($|\mathbf{c}_{\text{rest}}| = 0$).

Table 1: Weights w_q associated with the discrete velocities in the D2Q9 lattice.

w_R	w_U	w_L	w_D	w_{UR}	w_{UL}	w_{DL}	w_{DR}	w_{rest}
$\frac{1}{9}$				$\frac{1}{36}$				$\frac{4}{9}$

The evolution of f_q in a time step Δt is described by the lattice Boltzmann equation (LBE)

$$f_q(\mathbf{x}_n + \mathbf{c}_q \Delta t, t + \Delta t) = f_q(\mathbf{x}_n, t) + \Omega_q(\mathbf{x}_n, t), \quad (2.6)$$

where Ω_q is a collision operator. A widely used choice for Ω_q is the Bhatnagar–Gross–Krook (BGK) collision operator [BGK54]

$$\Omega_q(\mathbf{x}_n, t) = -\frac{\Delta t}{\tau} \left[f_q(\mathbf{x}_n, t) - f_q^{\text{eq}}(\mathbf{x}_n, t) \right], \quad (2.7)$$

where f_q^{eq} is the equilibrium distribution function and τ is a parameter which determines the rate of relaxation towards it, related to ν as

$$\nu = c_s^2 \left(\tau - \frac{\Delta t}{2} \right) \quad (2.8)$$

(as long as $\text{Ma} \ll 1$ and $\text{Ma} \ll \text{Re}$). Therefore, given a target value of Re , the relaxation time τ is fixed as

$$\tau = \frac{1}{c_s^2} \frac{U l}{\text{Re}} + \frac{\Delta t}{2}. \quad (2.9)$$

In *lattice units* where the lattice spacing $\Delta x = 1$ and the time step $\Delta t = 1$, adopting BGK collision operator reduces the LBE to

$$f_q(\mathbf{x}_n + \mathbf{c}_q, t + 1) = f_q(\mathbf{x}_n, t) - \frac{1}{\tau} \left[f_q(\mathbf{x}_n, t) - f_q^{\text{eq}}(\mathbf{x}_n, t) \right]. \quad (2.10)$$

The evolution in classical LBM is typically implemented in two steps: the *collision* step first updates the distribution function locally as

$$f_q^*(\mathbf{x}_n, t) = f_q(\mathbf{x}_n, t) - \frac{1}{\tau} \left[f_q(\mathbf{x}_n, t) - f_q^{\text{eq.}}(\mathbf{x}_n, t) \right], \quad (2.11)$$

and then the *streaming* step advects f_q^* to neighboring nodes as

$$f_q(\mathbf{x}_n + \mathbf{c}_q, t + 1) = f_q^*(\mathbf{x}_n, t). \quad (2.12)$$

Hereafter, we work in the lattice units. Note that the equilibrium distribution function is given by low-order terms in the expansion of the Maxwell–Boltzmann distribution, which can be written in the lattice units as

$$f_q^{\text{eq.}}(\mathbf{x}_n, t) = w_q \rho(\mathbf{x}_n, t) \left(1 + \frac{\mathbf{u}(\mathbf{x}_n, t) \cdot \mathbf{c}_q}{c_s^2} + \frac{(\mathbf{u}(\mathbf{x}_n, t) \cdot \mathbf{c}_q)^2}{2c_s^4} - \frac{\mathbf{u}(\mathbf{x}_n, t) \cdot \mathbf{u}(\mathbf{x}_n, t)}{2c_s^2} \right). \quad (2.13)$$

2.1.3 Implementation of boundary conditions

Periodic boundary conditions are straightforward to implement in LBM: during the streaming step, distribution functions that leave one side of the domain enter from the opposite side. In contrast, non-periodic boundaries require additional treatment.

No-slip stationary walls. We use the bounce-back scheme, in which distributions impinging on the wall are reflected in the opposite directions. For example, for a wall located halfway between lattice nodes on the bottom boundary, the bounce-back condition is

$$\begin{aligned} f_U(\mathbf{x}_n, t + 1) &= f_D^*(\mathbf{x}_n, t), \\ f_{UR}(\mathbf{x}_n, t + 1) &= f_{DL}^*(\mathbf{x}_n, t), \text{ and} \\ f_{UL}(\mathbf{x}_n, t + 1) &= f_{DR}^*(\mathbf{x}_n, t). \end{aligned} \quad (2.14)$$

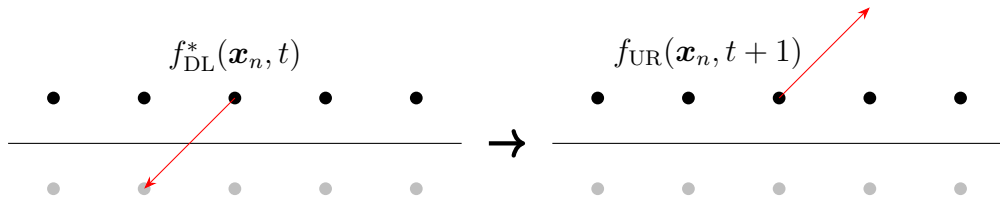


Figure 3: Halfway bounce-back boundary condition at a stationary wall.

Inflow. We impose a prescribed velocity using a moving-wall-type boundary condition. For example, let us consider an inflow at the left boundary with velocity $\mathbf{u}_{\text{in}}$. In this case, for a boundary located halfway between lattice nodes on the left boundary, we set

$$\begin{aligned} f_{\text{R}}(\mathbf{x}_n, t+1) &= f_{\text{L}}^*(\mathbf{x}_n, t) + \frac{2w_{\text{L}}\rho_{\text{in}}\mathbf{u}_{\text{in}} \cdot \mathbf{c}_{\text{L}}}{c_{\text{s}}^2}, \\ f_{\text{UR}}(\mathbf{x}_n, t+1) &= f_{\text{DL}}^*(\mathbf{x}_n, t) + \frac{2w_{\text{DL}}\rho_{\text{in}}\mathbf{u}_{\text{in}} \cdot \mathbf{c}_{\text{DL}}}{c_{\text{s}}^2}, \text{ and} \\ f_{\text{DR}}(\mathbf{x}_n, t+1) &= f_{\text{UL}}^*(\mathbf{x}_n, t) + \frac{2w_{\text{UL}}\rho_{\text{in}}\mathbf{u}_{\text{in}} \cdot \mathbf{c}_{\text{UL}}}{c_{\text{s}}^2}, \end{aligned} \quad (2.15)$$

where ρ_{in} is the density at the boundary node. Note that Eq. (2.15) specifies, after the streaming step, only the distribution functions whose velocities point into the interior of the fluid domain (right-pointing at the left boundary). These are the directions that would otherwise remain undefined, since they would require streaming from outside the fluid domain. The remaining directions at the same boundary node are obtained by ordinary streaming from interior nodes.

Outflow. We use a simple extrapolation from the interior.¹ For example, let us consider an outflow at the right boundary. For a boundary located halfway between lattice nodes on the right boundary, we set

$$\begin{aligned} f_{\text{L}}(\mathbf{x}_n, t+1) &= f_{\text{L}}(\mathbf{x}_n - \mathbf{c}_{\text{R}}, t+1), \\ f_{\text{UL}}(\mathbf{x}_n, t+1) &= f_{\text{UL}}(\mathbf{x}_n - \mathbf{c}_{\text{R}}, t+1), \text{ and} \\ f_{\text{DL}}(\mathbf{x}_n, t+1) &= f_{\text{DL}}(\mathbf{x}_n - \mathbf{c}_{\text{R}}, t+1). \end{aligned} \quad (2.16)$$

As in the inflow case, Eq. (2.16) specifies, after the streaming step, only the distribution functions whose velocities point into the interior of the fluid domain (left-pointing at the right boundary), with the remaining directions at the same boundary node obtained by ordinary streaming from interior nodes.

¹ Note that this outflow treatment is not commonly used in classical LBM simulations which use more accurate methods. However, it is equivalent to the zeroth-order Neumann boundary condition adopted in [JKL+25b] for the outlet, and is sufficient for the purpose of this study.

2.2 Quantum lattice Boltzmann method (QLBM)

In this section, we describe the quantum lattice Boltzmann method using a quantum linear system algorithm, developed in [LYW⁺23, PKJ⁺24, JKL⁺25a, JKL⁺25b]. While we largely adopt the formulation developed by the latter two, we depart from their choice of the shifted distribution function $g_q := f_q - w_q$ ² and instead work with the standard distribution function f_q throughout this paper. This choice is sufficient for the first-order Carleman linearization.

2.2.1 LBE in vector matrix form

Hereafter, we adopt the following notation for the velocity and node indices that appear in the collision-streaming pipeline:

- q, n : *pre-collision* (input) velocity and node indices.
- q^* : *post-collision* velocity index. Nodes are unchanged by collision.
- $q_{\text{out}}, n_{\text{out}}$: *post-streaming* (output) velocity and node indices.

Primes ' are reserved exclusively for additional summation dummy indices (all on the pre-collision side).

Let us rewrite the equilibrium distribution function (2.13) in terms of the distribution functions f . By assuming a weakly compressible flow ($1 - \rho \ll 1$), the inverse density is approximated as

$$\frac{1}{\rho} = (1 - (1 - \rho))^{-1} \approx 2 - \rho.$$

Substituting Eqs. (2.3) and (2.4) into Eq. (2.13), the equilibrium distribution function can

² Note that we are using the *shifted* variables from the beginning and their $\bar{f}_m$ corresponds to our f_q .

be expressed as

$$\begin{aligned}
& f_{q^*}^{\text{eq.}}(\mathbf{x}_n, t) \\
& \approx w_{q^*} \left[\begin{aligned} & \sum_q f_q(\mathbf{x}_n, t) \\ & + \sum_q f_q(\mathbf{x}_n, t) \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q^*})}{c_s^2} \\ & + \sum_{q, q'} f_q(\mathbf{x}_n, t) f_{q'}(\mathbf{x}_n, t) \left(2 - \sum_{q''} f_{q''}(\mathbf{x}_n, t) \right) \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q^*})(\mathbf{c}_{q'} \cdot \mathbf{c}_{q^*})}{2c_s^4} \\ & - \sum_{q, q'} f_q(\mathbf{x}_n, t) f_{q'}(\mathbf{x}_n, t) \left(2 - \sum_{q''} f_{q''}(\mathbf{x}_n, t) \right) \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q'})}{2c_s^2} \end{aligned} \right] \\
& = w_{q^*} \left[\begin{aligned} & \sum_q f_q(\mathbf{x}_n, t) \left(1 + \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q^*})}{c_s^2} \right) \\ & + \sum_{q, q'} f_q(\mathbf{x}_n, t) f_{q'}(\mathbf{x}_n, t) \left(\frac{(\mathbf{c}_q \cdot \mathbf{c}_{q^*})(\mathbf{c}_{q'} \cdot \mathbf{c}_{q^*})}{c_s^4} - \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q'})}{c_s^2} \right) \\ & + \sum_{q, q', q''} f_q(\mathbf{x}_n, t) f_{q'}(\mathbf{x}_n, t) f_{q''}(\mathbf{x}_n, t) \left(- \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q^*})(\mathbf{c}_{q'} \cdot \mathbf{c}_{q^*})}{2c_s^4} + \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q'})}{2c_s^2} \right) \end{aligned} \right].
\end{aligned} \tag{2.17}$$

We now vectorize the distribution functions to obtain a matrix-vector form of the discrete LBE. We define a vector $\mathbf{f}(t) \in \mathbb{R}^{NQ}$ by stacking $f_q(\mathbf{x}_n, t)$ as $f(t)[nQ + q] = f_q(\mathbf{x}_n, t)$, i.e.

$$\mathbf{f}(t) = \begin{pmatrix} f_0(\mathbf{x}_0, t) \\ \vdots \\ f_{Q-1}(\mathbf{x}_0, t) \\ f_0(\mathbf{x}_1, t) \\ \vdots \\ f_{Q-1}(\mathbf{x}_1, t) \\ \vdots \\ \vdots \\ f_{Q-1}(\mathbf{x}_{N-1}, t) \end{pmatrix}. \tag{2.18}$$

Using the expanded equilibrium distribution (2.17), the collision step (2.11) can be written in matrix-vector form as

$$\mathbf{f}^*(t) = (I + F_1)\mathbf{f}(t) + F_2\mathbf{f}(t)^{\otimes 2} + F_3\mathbf{f}(t)^{\otimes 3}, \tag{2.19}$$

where $\otimes$ denotes the Kronecker product. Since the collision operator acts locally at each lattice node, the nonzero elements of $F_1 \in \mathbb{R}^{NQ \times NQ}$, $F_2 \in \mathbb{R}^{NQ \times (NQ)^2}$, and $F_3 \in \mathbb{R}^{NQ \times (NQ)^3}$ are given as follows:

$$\begin{aligned}
(F_1)_{(nQ+q^*)(nQ+q)} &= \frac{1}{\tau} \left(-\delta_{q^*q} + w_{q^*} \left(1 + \frac{\mathbf{c}_q \cdot \mathbf{c}_{q^*}}{c_s^2} \right) \right) \\
(F_2)_{(nQ+q^*)[(nQ+q)NQ+(nQ+q')]} &= \frac{1}{\tau} w_{q^*} \left(\frac{(\mathbf{c}_q \cdot \mathbf{c}_{q^*})(\mathbf{c}_{q'} \cdot \mathbf{c}_{q^*})}{c_s^4} - \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q'})}{c_s^2} \right) \\
(F_3)_{(nQ+q^*)[(nQ+q)(NQ)^2+(nQ+q')NQ+(nQ+q'')]} &= \frac{1}{\tau} w_{q^*} \left(-\frac{(\mathbf{c}_q \cdot \mathbf{c}_{q^*})(\mathbf{c}_{q'} \cdot \mathbf{c}_{q''})}{2c_s^4} + \frac{(\mathbf{c}_q \cdot \mathbf{c}_{q'})}{2c_s^2} \right).
\end{aligned} \tag{2.20}$$

The streaming step (2.12) advects the post-collision distribution to neighboring nodes. In matrix-vector form, this is written as

$$\mathbf{f}(t+1) = S\mathbf{f}^*(t), \tag{2.21}$$

where S is a permutation matrix. The matrix S is defined so that $S_{(n_{\text{out}}Q+q_{\text{out}})(nQ+q^*)} = 1$ (and all other entries zero), where the output indices $(n_{\text{out}}, q_{\text{out}})$ depend on the boundary condition at node $\mathbf{x}_n$ for post-collision direction q^* . Corresponding to the boundary conditions in Sec. 2.1, the cases are

$$(\mathbf{x}_{n_{\text{out}}}, q_{\text{out}}) = \begin{cases} (\mathbf{x}_n + \mathbf{c}_{q^*}, q^*) & \text{(interior/periodic)} \\ (\mathbf{x}_n, \bar{q}^*) & \text{(bounce-back (2.14))} \\ (\mathbf{x}_n, \bar{q}^*) & \text{(inflow (2.15))} \\ (\mathbf{x}_n + \mathbf{c}_{q^*} + \mathbf{c}_R, q^*) & \text{(outflow (2.16))} \end{cases} \tag{2.22}$$

where $\bar{q}$ denotes the index of the velocity opposite to $\mathbf{c}_q$.

In addition to the streaming, the inflow boundary condition (2.15) introduces a velocity-dependent correction proportional to the local density $\rho_{\text{in}} = \sum_q f_q(\mathbf{x}_n, t)$. In general, this correction is linear in $\mathbf{f}$ and can be absorbed into the streaming-collision matrix. However, for the quantum circuit implementation, it is more convenient to approximate $\rho_{\text{in}} \approx 1$ and treat the inflow correction as a constant forcing. Under this approximation, the inflow boundary is handled by the same bounce-back mechanism as the no-slip walls, and the constant velocity corrections

$$b_{n_{\text{out}}Q+q_{\text{out}}} = \frac{2w_{q_{\text{out}}} \mathbf{u}_{\text{in}} \cdot \mathbf{c}_{q_{\text{out}}}}{c_s^2} \tag{2.23}$$

are grouped into a forcing vector $\mathbf{b} \in \mathbb{R}^{NQ}$, with $b_{n_{\text{out}}Q+q_{\text{out}}} = 0$ for nodes not on the inflow boundary or for non-outgoing directions. We adopt this approximation hereafter.

Combining the collision, boundary correction, and streaming steps, the discrete LBE in matrix-vector form reads

$$\mathbf{f}(t+1) = S \left[(I + F_1) \mathbf{f}(t) + F_2 \mathbf{f}(t)^{\otimes 2} + F_3 \mathbf{f}(t)^{\otimes 3} \right] + \mathbf{b}. \quad (2.24)$$

2.2.2 Carleman linearization

Implementing Eq. (2.24) on a quantum computer requires a linear formulation. The approach we adopt here is the Carleman linearization [Car32], which embeds the nonlinear recurrence into a linear recurrence on an extended space. Let us define the (infinite-dimensional) Carleman vector by

$$\mathbf{y}^{(\infty)}(t) := \begin{pmatrix} \mathbf{f}(t) \\ \mathbf{f}(t)^{\otimes 2} \\ \mathbf{f}(t)^{\otimes 3} \\ \vdots \end{pmatrix} \quad (2.25)$$

and similarly $\mathbf{y}^{*(\infty)}(t)$ by stacking $\mathbf{f}^*(t)^{\otimes k}$'s. Then, the collision step (2.19) can be represented as a linear map

$$\mathbf{y}^{*(\infty)}(t) = \mathcal{C}^{(\infty)} \mathbf{y}^{(\infty)}(t) \quad (2.26)$$

where the concrete form of the Carleman collision matrix $\mathcal{C}^{(\infty)}$ is derived by expanding each $\mathbf{f}^*(t)$ on the left-hand side. Similarly, the streaming step (2.21) and the constant forcing extend to the Carleman vector as

$$\mathbf{y}^{(\infty)}(t+1) = \mathcal{S}^{(\infty)} \mathbf{y}^{*(\infty)}(t) + \mathbf{b}_C^{(\infty)}, \quad (2.27)$$

where $\mathcal{S}^{(\infty)}$ is the block-diagonal streaming matrix

$$\mathcal{S}^{(\infty)} = \begin{pmatrix} S & & & \\ & S^{\otimes 2} & & \\ & & S^{\otimes 3} & \\ & & & \ddots \end{pmatrix}, \quad (2.28)$$

and $\mathbf{b}_C^{(\infty)}$ is the Carleman-extended forcing vector whose k -th block collects the terms involving $\mathbf{b}$ that arise when taking the k -fold Kronecker product of $\mathbf{f}(t+1)$.

In practice, we truncate the Carleman vector at a finite order N_C and reduce it to a d_C -dimensional vector ($d_C := \sum_{k=1}^{N_C} (NQ)^k$)

$$\mathbf{y}^{(N_C)}(t) := \begin{pmatrix} \mathbf{f}(t) \\ \mathbf{f}(t)^{\otimes 2} \\ \vdots \\ \mathbf{f}(t)^{\otimes N_C} \end{pmatrix}.$$

Correspondingly, the linear recurrence is also truncated as

$$\mathbf{y}^{(N_C)}(t+1) = \underbrace{\mathcal{S}^{(N_C)} \mathcal{C}^{(N_C)}}_{=: A^{(N_C)}} \mathbf{y}^{(N_C)}(t) + \mathbf{b}_C^{(N_C)}, \quad (2.29)$$

where the collision-streaming matrix $A^{(N_C)} \in \mathbb{R}^{d_C \times d_C}$. Furthermore, to improve the numerical stability of the time evolution, we introduce a step-size parameter $h \in [0, 1]$ and replace the recurrence (2.29) with the interpolated update

$$\begin{aligned} \mathbf{y}^{(N_C)}(t+h) &= (1-h)\mathbf{y}^{(N_C)}(t) + h(A^{(N_C)}\mathbf{y}^{(N_C)}(t) + \mathbf{b}_C^{(N_C)}) \\ &= \tilde{A}^{(N_C)}\mathbf{y}^{(N_C)}(t) + h\mathbf{b}_C^{(N_C)} \end{aligned} \quad (2.30)$$

where

$$\tilde{A}^{(N_C)} := (1-h)I + hA^{(N_C)}. \quad (2.31)$$

This interpolation suppresses patchy inflow transients arising in the linearized dynamics.³

For the first-order truncation ($N_C = 1$), the Carleman embedding is equivalent to keeping only the linear term in Eq. (2.24), and the collision-streaming matrix reduces to

$$A^{(1)} = S(I + F_1). \quad (2.32)$$

For each node n , input velocity index q , and post-collision velocity index q^* , the matrix element $A^{(1)}_{(n_{\text{out}}Q+q_{\text{out}})(nQ+q)}$ is determined by the boundary condition type at node n , where $(n_{\text{out}}, q_{\text{out}})$ denotes the streaming destination of (n, q^*) as defined in Eq. (2.22).

³ The update (2.30) advances the state by the fractional step h rather than a full lattice time unit. After N_t updates the physical time elapsed is $T = N_t h$. The choice $h = 1$ recovers the original recurrence (2.29).

2.2.3 Time evolution and final state idling

To compute $\mathbf{y}(N_th)$ on a quantum computer, we unroll the recurrence (2.30) into a single linear system following the approach of [Ber10, JKL⁺25a] which at the same time amplifies the probability of extracting the final state upon measurement.

The recurrence is first unrolled into $N_t + 1$ block rows (*evolution* steps). Then, to boost the probability of extracting the final state, $(2^W - 1)N_t - 1$ additional *idling* steps are appended. During the idling phase, the state is simply copied. The corresponding linear system is

$$\underbrace{\begin{pmatrix} I & & & & \\ -\tilde{A}^{(N_C)} & I & & & \\ & \ddots & \ddots & & \\ & & -\tilde{A}^{(N_C)} & I & \\ \hline & & & -I & I \\ & & & & \ddots & \ddots \\ & & & & & -I & I \end{pmatrix}}_{=:L} \underbrace{\begin{pmatrix} \mathbf{y}^{(N_C)}(0) \\ \mathbf{y}^{(N_C)}(h) \\ \vdots \\ \mathbf{y}^{(N_C)}(N_th) \\ \mathbf{y}^{(N_C)}(N_th) \\ \vdots \\ \mathbf{y}^{(N_C)}(N_th) \end{pmatrix}}_{=:b_L} = \underbrace{\begin{pmatrix} \mathbf{y}^{(N_C)}(0) \\ h\mathbf{b}_C^{(N_C)} \\ \vdots \\ h\mathbf{b}_C^{(N_C)} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{pmatrix}}_{=:b_L}. \quad (2.33)$$

Here, L is a $2^W N_t d_C \times 2^W N_t d_C$ block bidiagonal matrix whose block rows 2 to $N_t + 1$ have $-\tilde{A}^{(N_C)}$ (evolution) on the subdiagonal, while block rows $N_t + 2$ to $2^W N_t$ have $-I$ (idling) on the subdiagonal. The right-hand side contains the initial condition $\mathbf{y}^{(N_C)}(0)$ in the first block, the scaled forcing $h\mathbf{b}_C^{(N_C)}$ in blocks 2 to $N_t + 1$ (evolution), and zeros in the remaining blocks (idling).

Solving this linear system, one obtains a vector $\mathbf{y} = L^{-1}\mathbf{b}_L$ whose first $N_t + 1$ blocks encode the full time history $\mathbf{y}^{(N_C)}(0), \mathbf{y}^{(N_C)}(h), \dots, \mathbf{y}^{(N_C)}(N_th)$, and the remaining $(2^W - 1)N_t - 1$ blocks all contain the final state $\mathbf{y}^{(N_C)}(N_th)$. As a result, the final state $\mathbf{y}^{(N_C)}(N_th)$ occupies a fraction $(2^W - 1)/2^W$ of the solution vector $\mathbf{y}$, which approaches 1 exponentially with increasing W . When carried out on a quantum computer with sufficiently large W , this ensures that $\mathbf{y}^{(N_C)}(N_th)$ can be extracted with high probability upon measurement.

2.3 Quantum Circuit

The linear system (2.33) can be solved on a quantum computer using a quantum linear system algorithm (QLSA); see, e.g., [MPS⁺24] for a comprehensive survey of QLSAs. In this section, we describe the quantum-circuit implementation of QLBM using a QLSA based on the quantum singular value transformation (QSVT) [GSLW18].

2.3.1 State vector and velocity encoding

The vector $\mathbf{f}$ (2.18) is encoded in a quantum state on $n_{N_x} + n_{N_y} + n_Q$ qubits as

$$\frac{1}{\|\mathbf{f}\|} \sum_{n_x, n_y} \sum_{q=0}^{\tilde{Q}-1} f_q(\mathbf{x}_{n_x, n_y}) |n_x\rangle |n_y\rangle |q\rangle, \quad (2.34)$$

where $n_{N_x} = \lceil \log_2 N_x \rceil$, $n_{N_y} = \lceil \log_2 N_y \rceil$, $n_Q = \lceil \log_2 Q \rceil$, and $\tilde{Q} = 2^{n_Q}$. The coefficients corresponding to out-of-range indices, namely those not included in the velocity set, are set to zero. For simplicity, we only consider the case where N_x and N_y are powers of two.

To encode the D2Q9 velocities on $n_Q = 4$ qubits, we use a binary encoding in which the lower two bits $q[1:0]$ represent the x -component and the upper two bits $q[3:2]$ represent the y -component:

$$q[1:0] = \begin{cases} 00_2 & c_x = 0, \\ 01_2 & c_x = -1, \\ 10_2 & c_x = +1, \end{cases} \quad q[3:2] = \begin{cases} 00_2 & c_y = 0, \\ 01_2 & c_y = -1, \\ 10_2 & c_y = +1. \end{cases} \quad (2.35)$$

For example, the velocity index corresponding to $\mathbf{c}_{UL}$ in Fig. 2 is encoded as $q[1:0] = 01_2$ and $q[3:2] = 10_2$, giving $q = 1001_2 = 9$. The nine active velocity indices under this encoding are $q \in \{0, 1, 2, 4, 5, 6, 8, 9, 10\}$, while $q \in \{3, 7, 11, 12, 13, 14, 15\}$ are non-physical padding indices. The opposite-direction mapping $q \rightarrow \bar{q}$ used in the bounce-back boundary reduces to swapping the two bits in each pair: $q[0] \leftrightarrow q[1]$ for the x -component and $q[2] \leftrightarrow q[3]$ for the y -component.

2.3.2 Block-encoding of matrix $A^{(1)}$

The matrix $A^{(1)}$ (2.32) is block-encoded in a unitary $U_{A^{(1)}}$ as

$$(\langle 0^a | \otimes I) U_{A^{(1)}} (|0^a\rangle \otimes I) = A^{(1)} / \alpha_A, \quad (2.36)$$

where a is the number of ancilla qubits and α_A is the subnormalization factor. Equivalently, for each input basis state $|n_x\rangle |n_y\rangle |q\rangle$ on the system register,

$$U_{A^{(1)}}(|n_x\rangle |n_y\rangle |q\rangle |0^a\rangle) = \frac{1}{\alpha_A} \sum_{n_{\text{out}}, q_{\text{out}}} A^{(1)}_{(n_{\text{out}}Q+q_{\text{out}})(nQ+q)} |n_{\text{out},x}\rangle |n_{\text{out},y}\rangle |q_{\text{out}}\rangle |0^a\rangle + |\perp\rangle, \quad (2.37)$$

where $|\perp\rangle$ is a state orthogonal to the subspace where ancilla qubits are $|0^a\rangle$.

From the Eqs. (2.20) and (2.22), one can see that the structure of $A^{(1)}$ has two key properties:

- The element value depends only on the velocity pair (q^*, q) and the boundary condition type, and does not depend on the node index n .
- The output node n_{out} depends only on (n, q^*) and the boundary condition type, and does not depend on the input velocity q .

Based on these properties, the block-encoding uses the following ancilla registers encoding

- post-collision velocity index q^* (n_Q qubits),
- boundary condition type BC (2 qubits),
- amplitude (1 qubit).

The block-encoding unitary is decomposed as follows using four oracle quantum circuits:

1. Apply Hadamard gates to prepare a uniform superposition over the q^* register.
2. **O_{setBC}** : Compute the boundary-condition type at node $\mathbf{x}_n$ for the post-collision velocity direction q^* .
3. **$O_{\text{collision}}$** : Encode the information of the collision matrix on the amplitude register.
4. **$O_{\text{streaming}}$** : Update the node register from n to n_{out} and, when applicable, the velocity register from q^* to q_{out} .
5. **O_{unsetBC}** : Uncompute the BC register based on the updated node and velocity registers.
6. Apply Hadamard gates to the q^* register.
7. Apply an X gate to the amplitude register so that the block-encoding condition projects onto $|0\rangle$.

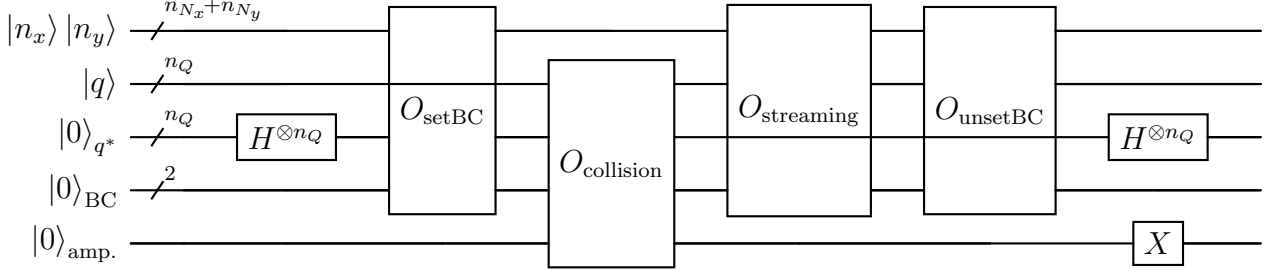


Figure 4: Quantum circuit for $U_{A^{(1)}}$.

O_{setBC} . This oracle computes one of the following three boundary-condition types:

- **Bounce-back** ($|\text{BC}\rangle = |11\rangle$) for
 - nodes at the left boundary ($n_x = 0$) with a left-pointing post-collision velocity.
 - nodes at the top ($n_y = N_y - 1$) or bottom ($n_y = 0$) boundaries with velocities pointing into the wall.
 - nodes adjacent to the obstacle with velocities pointing into it.

Since inflow uses the same bounce-back mechanism as no-slip walls, no separate inflow flag is needed.

- **Outflow** ($|\text{BC}\rangle = |10\rangle$) for
 - nodes at the right boundary ($n_x = N_x - 1$) with post-collision velocity being either right-pointing or non-physical ($q^* = 0011_2, 0111_2, 1011_2$).
- **Interior** ($|\text{BC}\rangle = |00\rangle$) otherwise.

These conditions are implemented using multi-controlled NOT gates on the BC register, conditioned on specific bit patterns of (n_x, n_y, q^*) .

$O_{\text{collision}}$. This oracle encodes the matrix elements C_{q^*q} of the collision matrix $I + F_1$ into the amplitude register via controlled- R_Y gates. The rotation angle for each pair (q^*, q) of active velocity indices (i.e. both in $\{0, 1, 2, 4, 5, 6, 8, 9, 10\}$) is

$$\theta_{q^*q} = 2 \arcsin\left(\frac{C_{q^*q}}{\max_{q^*,q} |C_{q^*q}|}\right), \quad (2.38)$$

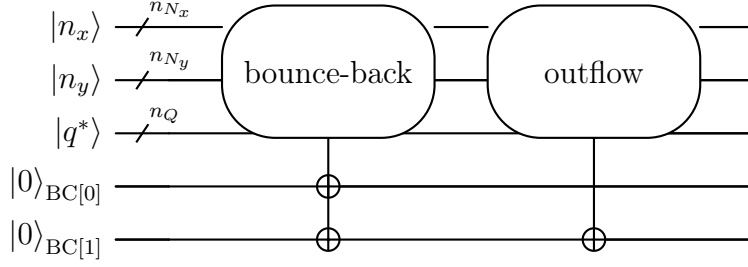


Figure 5: Quantum circuit for O_{setBC} . Each depicted gate actually consists of multiple multi-controlled NOT gates (controlled on each boundary condition).

where, recalling Eq. (2.20),

$$C_{q^*q} = \left(1 - \frac{1}{\tau}\right) \delta_{q^*q} + \frac{w_{q^*}}{\tau} \left(1 + \frac{\mathbf{c}_q \cdot \mathbf{c}_{q^*}}{c_s^2}\right). \quad (2.39)$$

When $C_{q^*q} < 0$, a controlled- Z gate needs to be additionally applied to correct the sign.

For the outflow boundary ($|\text{BC}\rangle = |10\rangle$, taken here as the right boundary), the Neumann condition requires that the outward ($c_x = +1$) populations at the boundary be discarded (they have no upstream source) and that the inward ($c_x = -1$) populations be supplied by extrapolation. To this end, additional R_Y gates load the extrapolation values at the non-physical post-collision indices $q^* \in \{3, 7, 11\}$ (i.e., $q^*[1:0] = 11_2$), with coefficients corresponding to the left-pointing velocities $q^* - 2 \in \{1, 5, 9\}$. The subsequent bit manipulation in $O_{\text{streaming}}$ then routes these values to the inward slot at the right boundary node.

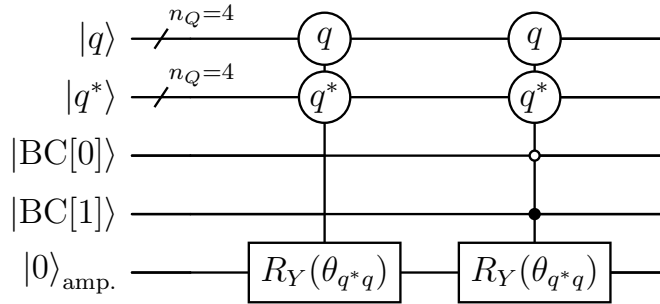


Figure 6: Quantum circuit for $O_{\text{collision}}$. Each depicted gate actually consists of multiple multi-controlled rotation gates.

$O_{\text{streaming}}$. This oracle first swaps the pre/post-collision velocity registers $|q\rangle$ and $|q^*\rangle$. The streaming operations are then applied to the pre-collision velocity register $|q^*\rangle_q$ (encoding the post-collision velocity q^* after the swap) and the node register, controlled on BC:

- **Interior** ($|BC\rangle = |00\rangle$): For each velocity component, a controlled adder (incrementer) or subtractor (decrementer) updates the corresponding coordinate register. Specifically, if $q^*[0] = 1$ (i.e., $c_x = -1$), subtract 1 from n_x ; if $q^*[1] = 1$ (i.e., $c_x = +1$), add 1 to n_x . The y -coordinate is updated analogously using $q^*[2]$ and $q^*[3]$.
- **Bounce-back** ($|BC\rangle = |11\rangle$): Controlled-SWAP gates exchange $q^*[0] \leftrightarrow q^*[1]$ (conditioned on $BC[1] = 1$) and $q^*[2] \leftrightarrow q^*[3]$ (conditioned on $BC[0] = 1$), mapping q^* to the opposite direction $\bar{q}^*$. The node register is unchanged.
- **Outflow** ($|BC\rangle = |10\rangle$): Building on the extrapolation values pre-loaded by $O_{\text{collision}}$ at $q^*[1:0] = 11_2$, the controlled-SWAP $0 \leftrightarrow 1$ (shared with bounce-back, conditioned on $BC[1] = 1$) followed by a controlled- X gate (conditioned on $BC = 10_2$) exchanging $q^*[1:0] = 10_2 \leftrightarrow 11_2$:
 - The extrapolation entries follow $11_2 \xrightarrow{\text{SWAP}} 11_2 \xrightarrow{X_1} 01_2$, becoming the boundary's inward ($c_x = -1$) component for the next time step.
 - The $q^*[1:0] = 10_2$ entries follow $10_2 \xrightarrow{\text{SWAP}} 01_2 \xrightarrow{X_1} 11_2$, parked at the non-physical index and thereby decoupled from the LBM dynamics.

The node register is updated for the y -coordinate as in the interior case, while the x -coordinate is unchanged. Concretely, the y -coordinate adder/subtractor is applied when $BC[0] = 0$ (i.e., $BC \in \{00_2, 10_2\}$), while the x -coordinate one is applied only when $BC = 00_2$.

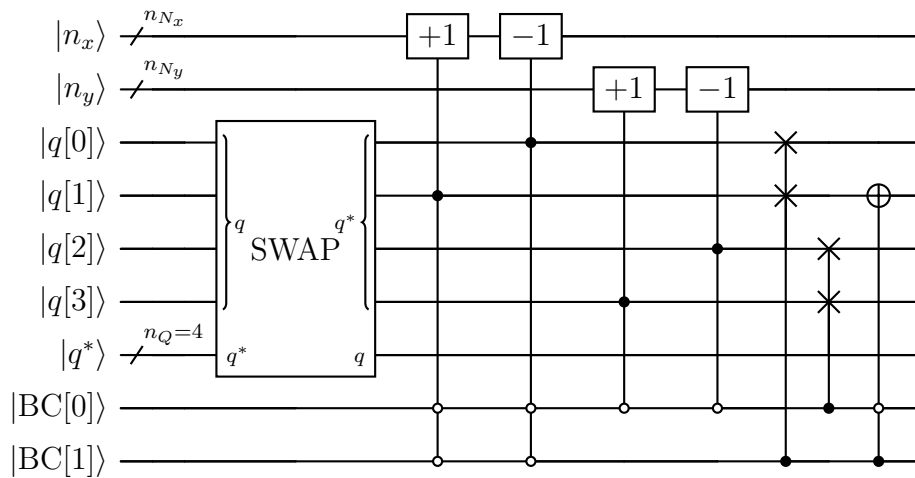


Figure 7: Quantum circuit for $O_{\text{streaming}}$.

O_{unsetBC} . This oracle uncomputes the BC register to $|00\rangle$. The quantum circuit has the same form as (inverse of) O_{setBC} except that O_{unsetBC} needs to control on the *updated* registers. For the streaming matrix S in Eq. (2.22), the boundary condition at $(n_{\text{out}}, q_{\text{out}})$ can be determined from $(n_{\text{out}}, q_{\text{out}})$ alone, so the uncomputation is well-defined. Specifically, the classification in terms of the output registers is:

- **Bounce-back** ($|\text{BC}\rangle = |11\rangle$): Since $O_{\text{streaming}}$ reverses the velocity without shifting the node, the output velocity points away from the boundary. This is for
 - nodes at the left boundary ($n_{\text{out},x} = 0$) with a right-pointing output velocity.
 - nodes at the top ($n_{\text{out},y} = N_y - 1$) or bottom ($n_{\text{out},y} = 0$) boundaries with velocities pointing into the interior.
 - nodes adjacent to the obstacle with velocities pointing away from it.
- **Outflow** ($|\text{BC}\rangle = |10\rangle$): Since the n_x shift is not applied for outflow, the node remains at the right boundary ($n_{\text{out},x} = N_x - 1$). Nodes at the right boundary with a left-pointing output velocity ($q_{\text{out}}[1:0] = 01_2$) and non-physical output velocity ($q_{\text{out}} = 0011_2, 0111_2, 1011_2$) are classified as outflow.
- **Interior** ($|\text{BC}\rangle = |00\rangle$): All other output index pairs.

2.3.3 Block-encoding of matrix L

The block-encoding unitary $U_{A(1)}$ is now used to construct a block-encoding unitary U_L . First, let us decompose the block-row index $l \in \{0, 1, \dots, 2^W N_t - 1\}$ as $l = s \cdot N_t + t$, where $N_t = 2^{n_t}$ is assumed. Then, it is encoded on a $(n_t + W)$ -qubit register $|t\rangle |s\rangle$ where

- $|t\rangle$: time-step register, indexing the time step within each phase.
- $|s\rangle$: phase register, distinguishing the evolution ($s = 0$) and idling ($s \geq 1$) phases.

Since L is block bidiagonal, each block row contains at most two nonzero blocks: the diagonal block I and the subdiagonal block $-\tilde{A}$ or $-I$ depending on the phase. Here we block-encode L as a sum of

- the diagonal I 's and the evolution-phase ($s = 0$) subdiagonal $-(1 - h)I$'s, and

- the evolution-phase ($s = 0$) subdiagonal $-hA^{(1)}$'s and the idling-phase ($s \geq 1$) subdiagonal $-I$'s

using an ancilla qubit $|\cdot\rangle_{\text{IN}_L}$ to distinguish between the two. Since $(1-h) \leq 1$, the maximum absolute value among the individual amplitudes is

$$c_L = \max\{1, \max_{q^*, q} h |C_{q^*q}|\}, \quad (2.40)$$

by which each rotation is normalized. Together with the factors due to the Hadamard gates acting on the q^* register and the summand-indicator IN_L register at both ends of the circuit, the overall subnormalization factor of the block-encoding is

$$\alpha_L = c_L \cdot (\sqrt{2}^{n_{Q+1}})^2. \quad (2.41)$$

The block-encoding of L proceeds as follows:

1. Apply Hadamard gates to the q^* and IN_L registers.
2. **Diagonal term:** Apply a controlled- R_Y gate to the amplitude register encoding the value $+1/c_L$, conditioned on $|0\rangle_{\text{IN}_L} |q_0^*\rangle_{q^*}$ for an arbitrarily chosen index q_0^* .
3. **Evolution subdiagonal, $-(1-h)I$ term:** Apply a controlled- R_Y gate to the amplitude register encoding the value $-(1-h)/c_L$, conditioned on $|0\rangle_{\text{IN}_L} |q_1^*\rangle_{q^*} |0\rangle_s$ for an arbitrarily chosen index q_1^* satisfying $q_1^* \neq q_0^*$.
4. **Evolution subdiagonal, $-hA^{(1)}$ term:** Controlled on $|1\rangle_{\text{IN}_L} |0\rangle_s$, apply the block-encoding $U_{A^{(1)}}$ to the system register, which encodes hC_{q^*q}/c_L into the amplitude.
5. **Idling subdiagonal, $-I$ term:** Apply a controlled- R_Y gate to the amplitude register encoding the value $-1/c_L$, conditioned on $|1\rangle_{\text{IN}_L} |q_2^*\rangle_{q^*} |s\rangle_s$ for any $s \geq 1$ and for an arbitrarily chosen index q_2^* . The last block row ($s = 2^W - 1$, $t = N_t - 1$) is excluded from this term using an auxiliary qubit.
6. **Row shift:** Increment the block-row index $l \rightarrow l + 1$ to map the subdiagonal column index to the correct row.
7. Apply Hadamard gates to the q^* and IN_L registers.
8. Apply an X gate to the amplitude register.

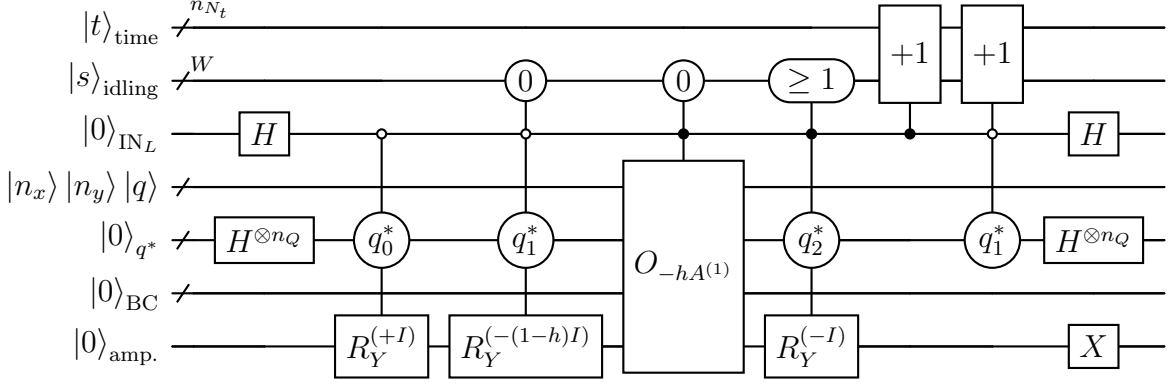


Figure 8: Quantum circuit for U_L . Here, $O_{-hA^{(1)}}$ is the inner part of the block-encoding circuit $U_{A^{(1)}}$ in Fig. 4, i.e., the sequence of O_{setBC} , $O_{\text{collision}}$, $O_{\text{streaming}}$, and O_{unsetBC} , with $O_{\text{collision}}$ modified to encode $-hC_{q^*q}/c_L$ in place of $C_{q^*q}/\max_{q^*,q} |C_{q^*q}|$. The control indices q_0^* , q_1^* , and q_2^* of the R_Y gates can be chosen arbitrarily as long as $q_0^* \neq q_1^*$.

2.3.4 QLSA by QSVT

To solve the linear system of equations (2.33), we employ a QLSA based on QSVT [GSLW18]. For a block-encoded matrix M with the singular value decomposition $V\Sigma W^\dagger$, QSVT by an odd (resp. even) polynomial P realizes a block-encoding of $P_{\text{SV}}(M) := VP(\Sigma)W^\dagger$ (resp. $VP(\Sigma)V^\dagger$). A circuit which achieves this consists of alternating applications of the block-encoding unitary and its inverse, interleaved with controlled phase rotations corresponding to the polynomial P , as shown in Fig. 9.

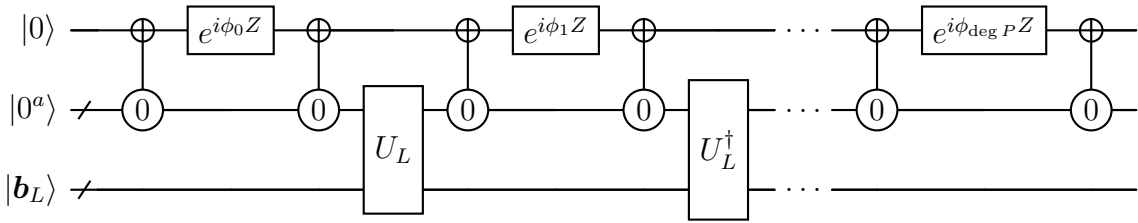


Figure 9: A quantum circuit for the QSVT-based QLSA. The ancilla register $|0^a\rangle$ includes all ancilla qubits used in the block-encoding of L (i.e., q^* , IN_L , BC , and the amplitude registers in our case).

If one chooses an odd polynomial $P_{\text{inv.}}$ approximating $1/(\kappa_{\text{eff.}} x)$ on $[1/\kappa_{\text{eff.}}, 1]$ such that the range contains all the singular values of the (subnormalized) block-encoded matrix,

QSVT (approximately) achieves matrix inversion and solves the linear system. The degree d_{QSVT} of the polynomial required to achieve an approximation error of at most ϵ_{QSVT} is known to be

$$d_{\text{QSVT}} = \mathcal{O}(\kappa_{\text{eff.}} \log \left(\frac{\kappa_{\text{eff.}}}{\epsilon_{\text{QSVT}}} \right)), \quad (2.42)$$

and this determines the number of queries to the block-encoding unitary.⁴

For the case of QLBM (2.33), if the approximation parameter $\kappa_{\text{eff.}}$ is chosen so that it satisfies

$$\kappa_{\text{eff.}} \geq \frac{1}{\sigma_{\min}(L/\alpha_L)}$$

where the right-hand side accounts for both the condition number of the original linear system and the block-encoding subnormalization factor, then the range $[1/\kappa_{\text{eff.}}, 1]$ contains all the singular values of the block-encoded matrix to be inverted by QSVT. According to [Ber10, Lem. 6],

$$\|L^{-1}\|_2 = \mathcal{O}(N_t \kappa_V) \quad (2.43)$$

where $\kappa_V = \|V\|_2 \|V^{-1}\|_2$ is the condition number of the matrix V that diagonalizes the evolution matrix $I - \mathcal{SC}$. Since the subnormalization factor α_L is independent of N_t , it follows that $\kappa_{\text{eff.}} \geq \alpha_L \|L^{-1}\|_2 = \mathcal{O}(\alpha_L N_t \kappa_V)$.

The output state, upon successful post-selection on the ancilla registers being $|0\rangle$, is proportional to $L^{-1} |\mathbf{b}_L\rangle$. The success probability of this post-selection is $\Omega(1/\kappa_{\text{eff.}}^2)$. In practice, when one uses quantum amplitude estimation to extract observables from the solution state, the estimation procedure internally handles the low success probability at the cost of an additional $\mathcal{O}(\kappa_{\text{eff.}})$ multiplicative overhead in the query complexity. It is therefore important to estimate $\kappa_{\text{eff.}}$ to evaluate the quantum resources required for the QLBM, which we will investigate in Sec. 3.1.

⁴ In this work, we used the polynomial introduced in [GKS21] and computed the angles based on [BS24].

3 Results

In this study, we consider two-dimensional flow around a rectangular obstacle in a square computational domain as a benchmark problem to evaluate the performance of the proposed QLBM. The number of grid points are set to powers of two with $N_x = N_y$ for simplicity, and an obstacle of size $N_x/8 \times N_y/4$ is placed in the upstream region of the channel, as shown in Fig. 10.

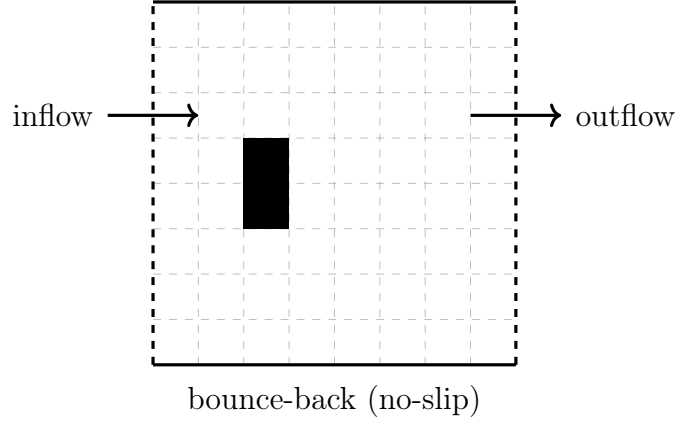


Figure 10: Flow configuration for the 2D channel with a rectangular obstacle. The gray dashed lines are added to indicate the size and position of the obstacle.

The common physical parameters are set as follows:

step-size parameter	: $h = 0.5$
Carleman truncation order	: $N_C = 1$
idling phase parameter	: $W = 1$
Reynolds number	: $\text{Re} = 1$ (based on the channel width)
Mach number of the inflow velocity	: $\text{Ma} = 0.01$

The initial condition is the fluid at rest (zero velocity everywhere, i.e. $f_q(\mathbf{x}_n, 0) \propto w_q$).

3.1 Effective condition number of the matrix L

We first compute the condition number and the maximum/minimum singular values of L for various grid sizes $N_x (= N_y)$ and time-step counts N_t . We take the channel width N_y (in lattice units) as the characteristic length l and $\text{Ma} \cdot c_s$ as the characteristic velocity U . The relaxation time τ is then determined by Eq. (2.9).

The results are shown in Fig. 11. As the total simulation time $T = N_t h$ increases, the condition number of L grows approximately linearly with T , which is due to the decrease of the minimum singular value. The growth of $\|L^{-1}\|$ with T is consistent with the bound in Eq. (2.43), $\|L^{-1}\| = O(N_t \kappa_V)$, since N_t is proportional to T and κ_V is independent of T . Furthermore, neither the spatial resolution N_x nor the presence of the obstacle significantly affects these spectral properties, confirming that T is the dominant driver of κ_{eff} . The effective condition number for a given problem instance is then obtained by multiplying $1/\sigma_{\min}(L)$ by the subnormalization factor α_L .

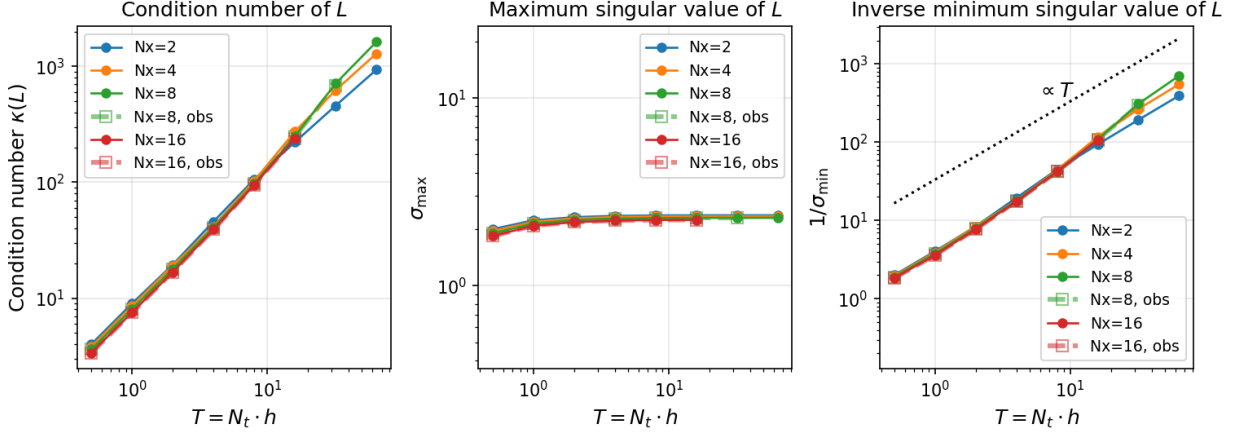


Figure 11: Spectral properties of the matrix L as a function of the total simulation time $T = N_t \cdot h$, for various grid sizes N_x ($= N_y$).

3.2 Simulation results of flow around an obstacle

We validate the proposed QLBM through two complementary approaches. First, we perform a state-vector simulation of the full quantum circuit on a small problem instance and compare it with a classical time-stepping simulation to confirm that the quantum algorithm correctly reproduces the linearized LBM dynamics. Second, we employ the Clenshaw algorithm to classically emulate the QSVT-based polynomial transformation. The state-vector simulation of the full quantum circuit requires simulating all qubits including the ancilla qubits used for block-encoding, which severely limits the tractable problem size. The Clenshaw algorithm operates directly on the matrix L without the ancilla overhead, enabling the same QSVT polynomial to be evaluated on significantly larger problem instances. This allows us to study larger N_t and the dependence of the solution accuracy on the polynomial degree d_{QSVT} and the effective condition number κ_{QSVT} .

3.2.1 Quantum circuit simulation

We perform a state-vector simulation of the full quantum circuit to verify that the quantum algorithm correctly reproduces the expected linearized LBM dynamics. Due to the exponential cost of state-vector simulation, we consider a small-scale problem with

- $N_x = N_y = 8$
- $N_t = 32$ ($T = 16$)
- $\kappa_{\text{QSVT}} = 3500$: chosen to exceed $1/\sigma_{\min}(L) \times \alpha_L \approx 3488$, where $1/\sigma_{\min}(L) \approx 109$ is read from Fig. 11 and $\alpha_L = 1 \cdot 2^5 = 32$ is the subnormalization factor of Eq. (2.41).
- $d_{\text{QSVT}} = 35001$: chosen based on Fig. 13 (presented later) to suppress the polynomial approximation error below 1%.

We compare the QLBM result with two classical references: (i) a classical time-stepping simulation of the linearized LBM (*linear reference*), and (ii) a classical time-stepping simulation of the original nonlinear LBM (*nonlinear reference*). Figure 12 shows the velocity fields at an intermediate time $t = 8$ and at the final time $t = T = 16$, and the 2-norm of the error as a function of time t . The QLBM result is in good agreement with the linear reference at all times, confirming that the quantum circuit correctly implements the intended linear system solver. Notably, the error of QLBM with respect to the linear reference remains well below that with respect to the nonlinear reference, suggesting that, in this particular case, the numerical error introduced by the QSVT-based polynomial approximation is negligible compared to the truncation error in the Carleman linearization.

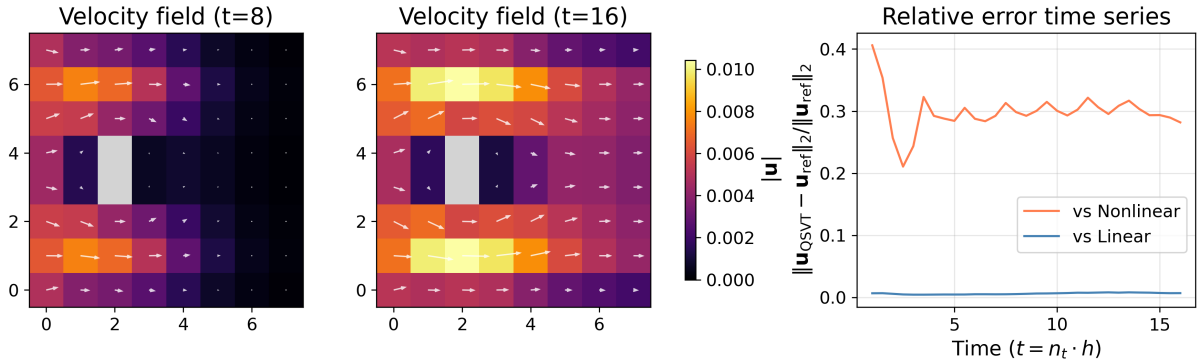


Figure 12: State-vector simulation results of the QLBM

($N_x = N_y = 8$, $N_t = 32$ ($T = 16$), $\kappa_{\text{QSVT}} = 3500$, $d_{\text{QSVT}} = 35001$).

3.2.2 Classical emulation via the Clenshaw algorithm

While the quantum circuit simulation validates the correctness of the QLBM, state-vector simulation of the full circuit requires exponential memory in the total number of qubits, including the numerous ancilla qubits introduced by the block-encoding construction. To circumvent this limitation and extend our analysis to problems with larger scale, we employ the Clenshaw algorithm [Cle55] to directly evaluate the polynomial transformation on the matrix L which is supposed to be realized by the QSVT quantum circuit.⁵

In the QSVT-based approach, the solution vector of the linear system $\mathbf{y} = L^{-1}\mathbf{b}_L$ is obtained by applying a polynomial transformation to the singular values of L/α_L :

$$\mathbf{y} \approx P_{\text{SV}}[L/\alpha_L] \mathbf{b}_L = \sum_{j=0}^{d_{\text{QSVT}}} a_j T_j^{\text{SV}} \left[\frac{L}{\alpha_L} \right] \mathbf{b}_L, \quad (3.1)$$

where T_j is the Chebyshev polynomial of degree j and $\{a_j\}$ are the expansion coefficients of the matrix-inversion polynomial $P(x) \approx 1/(\kappa_{\text{QSVT}} x)$ in the Chebyshev basis.

The Clenshaw algorithm evaluates the sum in Eq. (3.1) efficiently using the three-term recurrence relation of the Chebyshev polynomials. Since L is non-Hermitian, we work with the extended Hermitian matrix

$$H = \begin{pmatrix} 0 & L/\alpha_L \\ L^\dagger/\alpha_L & 0 \end{pmatrix} \quad (3.2)$$

and the extended vector $\mathbf{x} = (\mathbf{b}_L, \mathbf{0})^\top$. The Clenshaw recurrence defines a sequence of vectors

$$\mathbf{y}_j = \begin{cases} \mathbf{0} & (j = d_{\text{QSVT}} + 1) \\ a_{d_{\text{QSVT}}} \mathbf{x} & (j = d_{\text{QSVT}}) \\ 2H \mathbf{y}_{j+1} + a_j \mathbf{x} - \mathbf{y}_{j+2} & (\text{otherwise}) \end{cases} \quad (3.3)$$

and the result is given by $\sum_{j=0}^{d_{\text{QSVT}}} a_j T_j(H) \mathbf{x} = \mathbf{y}_0 - H \mathbf{y}_1$, whose lower half yields $P_{\text{SV}}[L/\alpha_L] \mathbf{b}_L$. Naively evaluating a degree- d_{QSVT} matrix polynomial would require d_{QSVT} matrix-matrix products, but the Clenshaw recurrence avoids this entirely. Each iteration requires only a

⁵ Note that an actual quantum circuit implementation additionally involves converting polynomial coefficients to QSVT phase angles and synthesizing the resulting rotation gates into Clifford+T gate sequences, each of which introduces its own approximation errors. The results presented here should therefore be understood as evaluations of the ideal polynomial transformation, not as predictions of the actual output of a physical quantum device.

single sparse matrix-vector product with H and vector additions, keeping the cost linear in d_{QSVT} and tractable even for large polynomial degrees.

Validation against the quantum circuit simulation. We first confirm that the Clenshaw algorithm reproduces the QSVT quantum circuit simulation results. Using the same problem size and parameters as in Sec. 3.2.1 ($N_x = N_y = 8$, $N_t = 32$), we compared the QSVT and Clenshaw outputs for several combinations of κ_{QSVT} and d_{QSVT} , including deliberately under-resolved settings where κ_{QSVT} lies below the estimated $\alpha_L/\sigma_{\min}(L)$.

In all cases, the Clenshaw-based solution agrees with the QSVT output to within a relative ℓ_2 error of 2×10^{-6} , even when the QSVT solution itself exhibits a relative error of several percent with respect to the linear LBM reference (e.g. $\kappa_{\text{QSVT}} = 3000$, $d_{\text{QSVT}} = 15001$). We note that this $\sim 10^{-6}$ residual, while far above machine precision, is consistent with the error introduced when converting polynomial coefficients to QSVT phase angles, which the Clenshaw algorithm bypasses by evaluating the polynomial directly from the coefficients. Since this level of discrepancy is negligible compared to the polynomial approximation error itself, the Clenshaw algorithm can be used as a proxy for the QSVT quantum algorithm, in the following larger-scale studies.

Table 2: Relative ℓ_2 errors between the outputs ($N_x = N_y = 8$, $N_t = 32$, $h = 0.5$).

κ_{QSVT}	d_{QSVT}	QSVT vs. Clenshaw	QSVT vs. linear LBM
3000	15001	2.1×10^{-7}	4.8×10^{-2}
3000	30001	8.2×10^{-7}	4.0×10^{-3}
3500	17501	3.6×10^{-7}	8.5×10^{-3}
3500	35001	1.7×10^{-6}	4.8×10^{-5}

Larger-scale simulation. Having validated the Clenshaw algorithm, we apply it to larger problem sizes $N_x = N_y = 16, 32$ with total simulation times $T = 16, 32, 64$. Here $T = N_t h$ is the *total* (maximum) simulation time, whereas $t = n_t h$ ($n_t = 0, 1, \dots, N_t$) denotes the *intermediate* time at step n_t . For each T , we choose two values of the condition number parameter κ_{QSVT} : a *low* value, set below $\alpha_L/\sigma_{\min}(L)$ estimated (and roughly extrapolated) from Fig. 11, and a *high* value, set above it. The accuracy is measured by the relative ℓ_2 -error between the Clenshaw-based solution and the linear reference, evaluated as a function of intermediate time t . We focus here on the most demanding case $T = 64$ ($N_t = 128$).

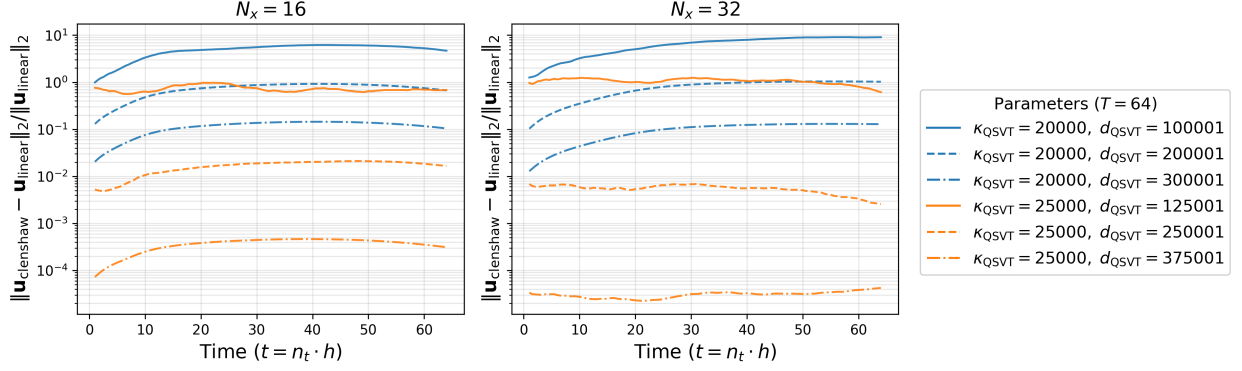


Figure 13: Accuracy of the Clenshaw-based solution with respect to the linear reference as a function of intermediate time t , with total simulation time $T = 64$.

As expected, when κ_{QSVT} is set below the true effective condition number, the error remains at the 10^{-1} level or above, regardless of d_{QSVT} . In contrast, when κ_{QSVT} exceeds the true value, the error drops to the 10^{-1} level or below for sufficiently large d_{QSVT} . Moreover, the rate at which the error decreases with increasing d_{QSVT} is substantially larger for the high- κ_{QSVT} cases, consistent with the polynomial approximation theory (2.42): when κ_{QSVT} is underestimated, increasing d_{QSVT} alone cannot compensate for the mismatch.

To visualize these effects in physical space, Fig. 14 compares the velocity fields at intermediate time for three representative parameter choices. When κ_{QSVT} is not large enough (Fig. 14a), the flow field is broadly correct in the interior but develops spurious artifacts near the outflow (right) boundary. When the polynomial degree is not large enough (Fig. 14b), the entire velocity field is corrupted by oscillatory noise. Only when both κ_{QSVT} and d_{QSVT} are sufficiently large (Fig. 14c) does the Clenshaw-based solution achieve good agreement with the linear reference.

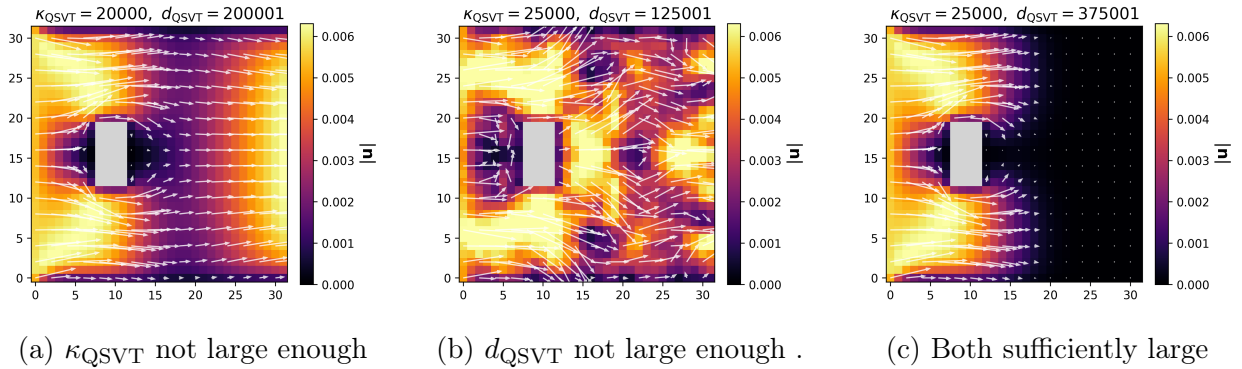


Figure 14: Velocity fields at $t = 25$ ($n_t = 50$) for $N_x = N_y = 32$, $T = 64$.

3.3 Quantum computational costs of U_L

We analyze the quantum computational costs of the block-encoding U_L by counting the required numbers of elementary gates and qubits for various $N_x (= N_y)$ and N_t . Figure 15a shows the count of each elementary gate for $N_t = 64$ as a function of $N_x = 2^{n_{N_x}}$. The number of Toffoli gates grows logarithmically with N_x , while those of the other gates remain nearly constant. Figure 15b further shows the Toffoli gate count for $N_t = 64, 128, 256$, which is almost independent of N_t . This is consistent with the structure of U_L , in which the time-step dependence enters only through a small number of operations on ancilla qubits. Figure 16 shows the total and ancilla qubit counts, both of which again grow logarithmically with N_x , in agreement with the asymptotic analysis of Sec. 2.

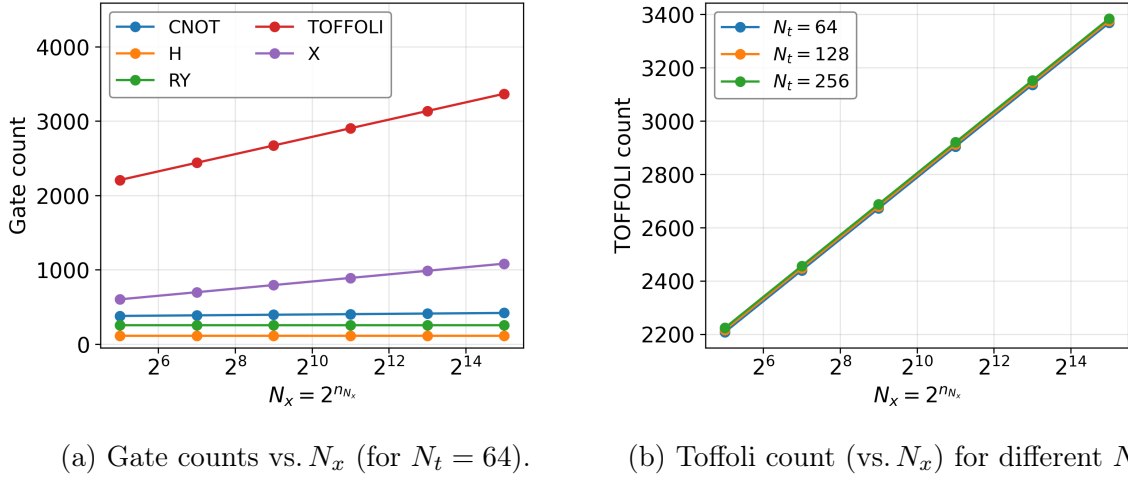


Figure 15: Quantum gate counts of U_L .

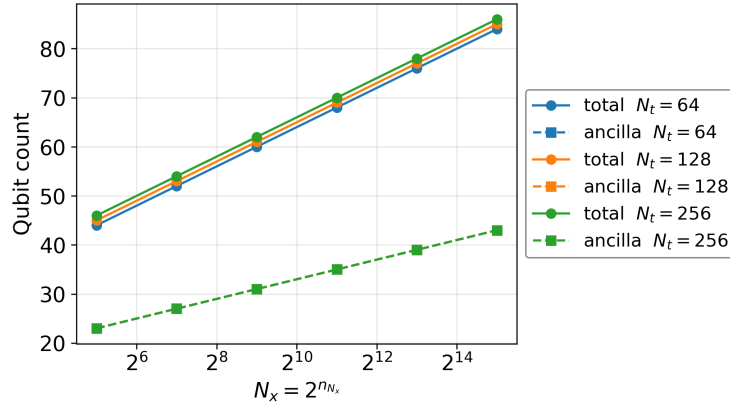


Figure 16: Qubit counts of U_L . The ancilla qubit counts are identical for all N_t values.

3.4 Fault-tolerant T-gate cost estimation

To translate the gate counts into fault-tolerant resource requirements, we roughly estimate the total number of T gates needed for a single application of the QSVT-based QLSA. Following [JKL⁺25a], we decompose

- each Toffoli gate (in U_L) into 7 T gates, and
- each R_Y gate (in U_L) and R_z gate (in QSVT circuit) into $3 \log_2(1/\epsilon_{\text{gate}})$ T gates via the Clifford+T approximation [RS14], where ϵ_{gate} is the per-gate synthesis precision.

Because the QSVT circuit calls U_L a total of d_{QSVT} times, where d_{QSVT} is the polynomial degree (2.42), the total T-gate count is

$$N_{\text{T}}^{(\text{total})} = N_{\text{Toffoli}}^{(U_L)} \cdot d_{\text{QSVT}} \cdot 7 + \left(N_{R_Y}^{(U_L)} \cdot d_{\text{QSVT}} + (d_{\text{QSVT}} + 1) \right) \cdot 3 \log_2 \left(\frac{1}{\epsilon_{\text{gate}}} \right) \quad (3.4)$$

where $N_{\text{Toffoli}}^{(U_L)}$ (resp. $N_{R_Y}^{(U_L)}$) is the Toffoli (resp. R_Y) gate counts of a single invocation of U_L (Fig. 15). For the polynomial degree, we use $d_{\text{QSVT}} = c \cdot \kappa_{\text{QSVT}} + 1$, dropping the logarithmic factor in Eq. (2.42) and absorbing it into the proportionality constant c . The effective condition number is approximated as $\kappa_{\text{QSVT}} \approx 4 \cdot T^{1.2} \cdot 32$, obtained by fitting $1/\sigma_{\min}(L)$ for the $N_x = 16$ obstacle configuration in Fig. 11 and multiplying the subnormalization factor α_L with an additional safety margin of 1.2. Because this fit is based on a single small grid size and is extrapolated to larger T values, it should be regarded as an order-of-magnitude approximation of κ_{eff} . The per-gate synthesis precision is set to $\epsilon_{\text{gate}} = \epsilon_{\text{base}}/d_{\text{QSVT}}$ to keep the total synthesis error bounded by ϵ_{base} . Based on the error analysis in Fig. 13, we set $c = 10$, i.e. $d_{\text{QSVT}} = 10\kappa_{\text{QSVT}} + 1$. Under this choice, the QSVT approximation error is on the order of a few percent, so we set $\epsilon_{\text{base}} = 0.01$ to keep the gate-synthesis error at a comparable level.

Figure 17 shows the resulting T-gate counts. The total T-gate count ranges from approximately 10^{10} to 10^{12} , depending primarily on T : each doubling of T increases the count by roughly an order of magnitude. In contrast, the dependence on N_x is weak: the curves are nearly flat, consistent with the logarithmic scaling of the gate cost (Fig. 15) and the finding that κ_{eff} is driven by T rather than N_x (Sec. 3.1). We stress that these estimates are order-of-magnitude figures that rely on several approximations, notably the extrapolated κ_{QSVT} fit, the choice of the proportionality constant c , and the asymptotic gate-synthesis

cost, and are intended to illustrate the relative scaling with T and N_x rather than to provide precise resource counts. Moreover, they do not include the additional overhead from amplitude estimation to be discussed in Sec. 4.

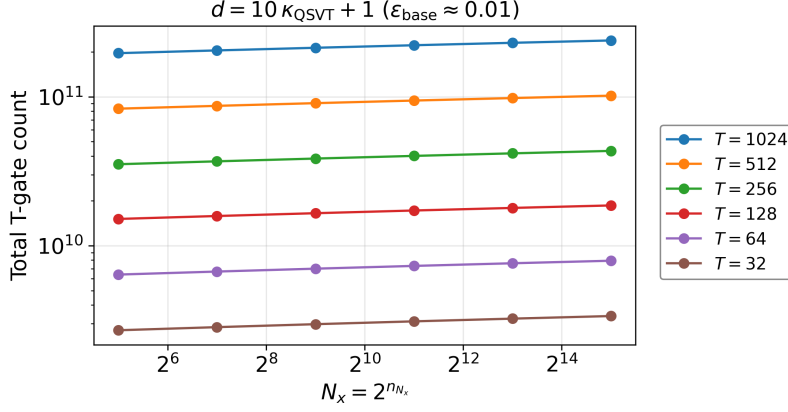


Figure 17: Estimated total T-gate count for a single QSVT-based linear system solve as a function of N_x , with $d_{\text{QSVT}} = 10 \kappa_{\text{QSVT}} + 1$. Under this setting, the QSVT approximation error and the total gate-synthesis error ($\epsilon_{\text{base}} = 0.01$) are both on the order of a few percent. The T-gate count is dominated by the simulation time T and depends only weakly on the spatial resolution N_x .

4 Discussion

Comparison with [JKL⁺25a, JKL⁺25b]. As discussed in Sec. 1, the present work extends the framework of [JKL⁺25a, JKL⁺25b] in two directions:

- (i) a characterization of how the effective condition number κ_{QSVT} (hence the number of queries to the block-encoding unitary) depends on N_t and N_x separately, and
- (ii) an explicit gate-level construction of the block-encoding of the system matrix that incorporates realistic boundary conditions and obstacle geometry.

Regarding (i), [JKL⁺25a] concluded that quantum advantage at high Reynolds numbers is at most polynomial over classical CFD. Our independent analysis (Sec. 3.1) provides a more detailed picture: κ_{QSVT} grows strongly with N_t but depends only weakly on N_x . At high Reynolds numbers, achieving a given physical simulation time requires resolving finer spatial and temporal scales, which translates to a larger N_t per advection time and thus a

polynomially growing κ_{QSVT} . Since the QSVT polynomial degree scales as $\tilde{\mathcal{O}}(\kappa_{\text{QSVT}})$, this drives up the overall quantum resource cost, consistent with their conclusion. In contrast, when N_t is fixed and the domain size (in effect N_x) is increased, the condition number remains nearly constant, so the quantum computational cost grows only mildly with system size.

Regarding (ii), the gate-level construction in Sec. 2.3 shows that the quantum computational cost scales as $\mathcal{O}(\log N_x)$, so the marginal cost of increasing spatial resolution is low. The fault-tolerant T-gate cost estimation in Sec. 3.4 quantitatively confirms this: the total T-gate count for a single QSVT-based solve is dominated by the simulation time T rather than N_x .

Together, these two observations suggest that the most favorable regime for the proposed algorithm is one in which a large spatial domain is simulated over a moderate number of time steps. Concrete examples include computing ensemble statistics by superposing runs with slightly different initial conditions, or exploring design variations, such as small perturbations to the obstacle geometry, within a single quantum computation. Even under the constraint of a limited number of time steps, the logarithmic scaling with N_x makes it possible to accommodate a large number of lattice points, or equivalently a large number of ensemble members, at modest additional quantum cost.

Algorithmic limitations and future improvements. The present implementation adopts the first-order Carleman truncation ($N_C = 1$), which linearizes the nonlinear LBE by retaining only the leading-order term. While this is sufficient for the low-Ma and low-Re regime, it introduces a truncation error that grows with the Mach number. Higher-order Carleman truncation ($N_C \geq 2$) can reduce this error at the cost of an exponential increase in the state-space dimension. The work of [JKL⁺25a] developed scalable block-encoding strategies and presented gate-cost estimates for arbitrary truncation orders N_C . Extending our circuit-level framework to support $N_C \geq 2$ is an important direction for future work. We note that for the first-order truncation adopted in this work ($N_C = 1$), the shifted-LBM equilibrium distribution introduced in [JKL⁺25a] is not required. However, at higher truncation orders, the shifted formulation becomes essential: it keeps the norm of the state vector bounded as $\mathcal{O}(1)$, thereby controlling the overhead of extracting classical information from the quantum state.

Applicability at high Reynolds numbers. A fundamental challenge for Carleman-linearization-based quantum algorithms is that the convergence of the Carleman series is not guaranteed at high Reynolds numbers. In the LBM setting, this translates to a restriction on the Mach number of the inflow, which must remain sufficiently small for the Carleman expansion to converge. The present study is conducted at $\text{Re} = 1$ and $\text{Ma} = 0.01$, well within this regime. Extending the algorithm to turbulent, high-Reynolds-number flows requires either higher-order Carleman truncation with provable convergence bounds or fundamentally different linearization strategies, and is left for future work.

End-to-end resource estimates. The fault-tolerant T-gate cost estimation in Sec. 3.4 provides a concrete resource estimate for the block-encoding-plus-QSVT portion of the algorithm. However, a complete end-to-end resource estimate must also account for several additional costs that are not addressed in this work. First, extracting flow observables such as the drag force from the quantum state requires quantum amplitude estimation (as described, for example, in [PKJ⁺24]), which introduces an additional multiplicative overhead of $\mathcal{O}(\kappa_{\text{QSVT}}/\epsilon)$ relative to a single application of the QLSA, where ϵ is the target precision. Combined with the QSVT polynomial degree $d_{\text{QSVT}} = \tilde{\mathcal{O}}(\kappa_{\text{QSVT}})$, the total number of queries to the block-encoding scales as $\tilde{\mathcal{O}}(\kappa_{\text{QSVT}}^2/\epsilon)$, which grows quadratically with N_t up to logarithmic factors. Second, the cost of preparing the initial quantum state encoding the initial condition $\mathbf{b}_L$ has not been included in our estimates. This cost depends on the structure of the initial data and may itself require nontrivial quantum circuits. For uniform initial conditions such as the zero-velocity state adopted in this work, however, state preparation is straightforward, as the same value is assigned to all lattice points and the preparation cost does not scale with N_x . Accounting for these readout and state-preparation costs would substantially increase the total quantum resource requirements beyond the estimates in Sec. 3.4 and should be incorporated in future end-to-end analyses.

5 Conclusion

In this work, we developed and validated a quantum algorithm for simulating two-dimensional flow around an obstacle based on Carleman-linearized LBM and QSVT-based matrix inversion. The central contribution is an explicit gate-level construction of the quantum circuit for the system matrix block-encoding U_L , which faithfully encodes the full time-discretized LBM dynamics—including inflow, outflow, and no-slip boundary conditions as well as an

obstacle geometry—without relying on high-level oracle abstractions. To our knowledge, this is the first demonstration of a QLBM circuit that incorporates realistic inflow and outflow boundary conditions at the gate level.

Furthermore, a systematic numerical study of the effective condition number κ_{QSVT} of the system matrix L reveals that κ_{QSVT} grows strongly with the number of time steps N_t , whereas its dependence on the number of lattice points N_x is comparatively weak. This finding identifies N_t as the dominant driver of quantum resource costs in the global-in-time formulation, and suggests that the proposed algorithm is most competitive in the regime of large spatial resolution but moderate temporal extent.

We validated the quantum algorithm through state-vector simulation and also confirmed that the Clenshaw algorithm faithfully emulates the QSVT polynomial transformation. The latter classical emulation allowed us to study larger problem sizes than are accessible to full state-vector simulation, and to directly observe how the choice of condition number parameter κ_{QSVT} and polynomial degree d_{QSVT} affects the accuracy of the reconstructed flow field. This provides a practical tool for calibrating quantum resource parameters before running the algorithm on actual quantum hardware.

Finally, a gate-count analysis of the block-encoding U_L shows that the total number of gates scales approximately logarithmically with N_x , in agreement with the asymptotic analysis. When translated into fault-tolerant T-gate costs, the total count ranges from 10^{10} to 10^{12} for the problem sizes considered ($N_x = N_y \leq 10^{16}$ and $T \lesssim 10^3$), with the simulation time T remaining the dominant cost driver rather than the spatial resolution N_x . This logarithmic scaling is encouraging for large-scale applications, as it implies that substantially increasing the spatial resolution of the simulation requires only a modest increase in quantum circuit complexity.

Acknowledgments

The authors thank Hayato Higuchi and Kouki Nakamura for helpful discussions. This work was performed using the JAXA Supercomputer System ‘JSS3’. This work was also supported by MEXT as “Program for Promoting Researches on the Supercomputer Fugaku” and used computational resources of supercomputer Fugaku provided by the RIKEN Center for Computational Science (Project ID: ra250010). The authors would like to thank Masafumi Yamazaki for his valuable support on the use of Fugaku supercomputer system.

A Notation

Lattice Boltzmann method

D	: spatial dimension
Q	: number of discrete velocities
N_x, N_y	: number of lattice nodes in each direction
N	: total number of lattice nodes ($N = N_x \times N_y$)
$\mathbf{x}_n$	: position of the n -th lattice node
$\mathbf{c}_q$	: discrete velocity vector for index q
w_q	: weight associated with $\mathbf{c}_q$
$f_q(\mathbf{x}_n, t)$	: discrete distribution function
$f_q^{\text{eq}}(\mathbf{x}_n, t)$	: equilibrium distribution function
$f_q^*(\mathbf{x}_n, t)$	: post-collision distribution function
$\rho(\mathbf{x}_n, t)$	: macroscopic density
$\mathbf{u}(\mathbf{x}_n, t)$	: macroscopic velocity
c_s	: lattice speed of sound ($= 1/\sqrt{3}$ in lattice units)
ν	: kinematic viscosity
τ	: relaxation time
Ω_q	: collision operator
$\bar{q}$	: index of the velocity opposite to $\mathbf{c}_q$
$\mathbf{u}_{\text{in}}$	: prescribed inflow velocity
ρ_{in}	: density at the boundary node

Matrix-vector formulation and Carleman linearization

$\mathbf{f} \in \mathbb{R}^{NQ}$	: vectorized distribution function
F_1, F_2, F_3	: collision matrices (linear, quadratic, cubic)
S	: streaming permutation matrix
$\mathbf{b} \in \mathbb{R}^{NQ}$	: inflow forcing vector
$A^{(1)} = S(I + F_1)$	: collision-streaming matrix (first-order)
q, n	: pre-collision velocity and node indices (input)
q^*	: post-collision velocity index (node unchanged by collision)
$q_{\text{out}}, n_{\text{out}}$	: post-streaming velocity and node indices (output)
C_{q^*q}	: collision matrix element of $I + F_1$

N_C	: Carleman truncation order
$\mathbf{y}(t)$	: Carleman vector (stacked tensor powers of $\mathbf{f}$)
d_C	: dimension of the Carleman vector
$\mathcal{C}, \mathcal{S}$	: Carleman collision and streaming matrices
$\mathbf{b}_C$	: Carleman-extended forcing vector
h	: step-size parameter for interpolated update
$\tilde{A}$	: interpolated evolution matrix, $(1 - h)I + hA^{(1)}$

Linear system

N_t	: number of time steps
T	: total simulation time ($= N_t \cdot h$)
W	: idling phase parameter
L	: block bidiagonal matrix of the linear system
$\mathbf{y}$	: solution vector
$\mathbf{b}_L$	: right-hand side vector

Quantum circuit

n_{N_x}, n_{N_y}	: number of qubits for spatial registers
n_Q	: number of qubits for velocity register
$\tilde{Q}$	: padded number of velocities ($= 2^{n_Q}$)
n_{N_t}	: number of qubits for time-step register
BC	: boundary-condition type
α_A	: subnormalization factor of $A^{(1)}$ block-encoding
α_L	: subnormalization factor of L block-encoding
c_L	: normalization factor for individual R_Y rotation amplitudes in U_L
κ_{QSVT}	: condition number parameter for QSVT polynomial
d_{QSVT}	: polynomial degree parameter for QSVT
$U_{A^{(1)}}$	: block-encoding unitary for $A^{(1)}$
U_L	: block-encoding unitary for L

References

- [Ber10] D. W. Berry, *High-order quantum algorithm for solving linear differential equations*, *J. Phys. A Math. Theor.* **47** (2014) 105301, [arXiv:1010.2745 \[quant-ph\]](#).
- [BGK54] P. L. Bhatnagar, E. P. Gross, and M. Krook, *A Model for Collision Processes in Gases. I. Small Amplitude Processes in Charged and Neutral One-Component Systems*, *Phys. Rev.* **94** (1954) 511–525.
- [BS24] B. K. Berntson and C. Sünderhauf, *Complementary polynomials in quantum signal processing*, *Commun. Math. Phys.* **406** (2025) , [arXiv:2406.04246 \[quant-ph\]](#).
- [Bud21] L. Budinski, *Quantum algorithm for the advection-diffusion equation simulated with the lattice Boltzmann method*, *Quantum Inf. Process.* **20** (2021) 1–17.
- [Car32] T. Carleman, *Application de la théorie des équations intégrales linéaires aux systèmes d'équations différentielles non linéaires*, *Acta Math.* **59** (1932) 63–87.
- [Cle55] C. W. Clenshaw, *A note on the summation of Chebyshev series*, *Math. Comp.* **9** (1955) 118–120.
- [GKS21] S. Gribling, I. Kerenidis, and D. Szilágyi, *An optimal linear-combination-of-unitaries-based quantum linear system solver*, *ACM Trans. Quantum Comput.* **5** (2024) 1–23, [arXiv:2109.04248 \[quant-ph\]](#).
- [GSLW18] A. Gilyén, Y. Su, G. H. Low, and N. Wiebe, *Quantum Singular Value Transformation and beyond: Exponential improvements for quantum matrix arithmetics*, *STOC 2019: Proc. 51st Ann. ACM SIGACT Symp. on Theory of Computing* (2019) 193–204, [arXiv:1806.01838 \[quant-ph\]](#).
- [JKL⁺25a] D. Jennings, K. Korzekwa, M. Lostaglio, R. Ashworth, E. Marsili, and S. Rolston, *An end-to-end quantum algorithm for nonlinear fluid dynamics with bounded quantum advantage*, [arXiv:2512.03758 \[quant-ph\]](#).
- [JKL⁺25b] D. Jennings, K. Korzekwa, M. Lostaglio, P. Mannix, R. Ashworth, E. Marsili, and S. Rolston, *Simulating non-trivial incompressible flows with a quantum lattice Boltzmann algorithm*, [arXiv:2512.05781 \[physics.flu-dyn\]](#).

- [KKK⁺16] T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva, and E. M. Viggen, *The Lattice Boltzmann Method: Principles and Practice*, Graduate Texts in Physics, Springer International Publishing, 2016.
- [LYW⁺23] X. Li, X. Yin, N. Wiebe, J. Chun, G. K. Schenter, M. S. Cheung, and J. Mülmenstädt, *Potential quantum advantage for simulation of fluid dynamics*, *Phys. Rev. Res.* **7** (2025) 013036, [arXiv:2303.16550 \[quant-ph\]](#).
- [MPS⁺24] M. E. S. Morales, L. Pira, P. Schleich, K. Koor, P. C. S. Costa, D. An, A. Aspuru-Guzik, L. Lin, P. Rebentrost, and D. W. Berry, *Quantum linear system solvers: A survey of algorithms and applications*. [arXiv:2411.02522 \[quant-ph\]](#).
- [MSL⁺15] A. Mezzacapo, M. Sanz, L. Lamata, I. L. Egusquiza, S. Succi, and E. Solano, *Quantum simulator for transport phenomena in fluid flows*, *Sci. Rep.* **5** (2015) 13153, [arXiv:1502.00515 \[quant-ph\]](#).
- [PKJ⁺24] J. Penuel, A. Katabarwa, P. D. Johnson, P. Kuklinski, B. Rempfer, C. Farquhar, Y. Cao, and M. C. Garrett, *Detailed assessment of calculating drag force with quantum computers: Explicit time-evolution precludes exponential advantage for nonlinear differential equations*, [arXiv:2406.06323 \[quant-ph\]](#).
- [RS14] N. J. Ross and P. Selinger, *Optimal ancilla-free Clifford+T approximation of z-rotations*, *Quantum Inf. Comput.* **16** (2016) 901–953, [arXiv:1403.2975 \[quant-ph\]](#).
- [SS23] C. Sanavio and S. Succi, *Lattice Boltzmann-Carleman quantum algorithm and circuit for fluid flows at moderate Reynolds number*, *AVS Quantum Sci.* **6** (2024) 023802, [arXiv:2310.17973 \[quant-ph\]](#).
- [SSR⁺25] C. Sanavio, W. A. Simon, A. Ralli, P. Love, and S. Succi, *Carleman-lattice-Boltzmann quantum circuit with matrix access oracles*, *Phys. Fluids* **37** (2025) 037123, [arXiv:2501.02582 \[quant-ph\]](#).
- [Suc18] S. Succi, *The Lattice Boltzmann Equation: For Complex States of Flowing Matter*, Oxford University Press, 2018.